Week 5

YARN, file formats and compression

How a shared Hadoop cluster hands out CPU and RAM to many jobs (YARN), and how the way data is laid out and compressed on disk decides how much of it a query has to read. Part 2 tests the same ideas on LinkedIn's 5,000-node YARN cluster.

Exam questions

Multiple choice (4)

Q1 Which component in YARN is responsible for global resource allocation across the entire cluster?

(a) NodeManager
(b) ApplicationMaster
(c) ResourceManager
(d) NameNode

> **(c) ResourceManager**
> One RM per cluster allocates containers globally; the AM manages only its own job.

Q2 Which Hadoop file format stores data in columnar format and is best suited for analytical, read-heavy workloads?

(a) Avro
(b) Text/CSV
(c) Parquet
(d) JSON

> **(c) Parquet**
> Columnar, with min/max stats for predicate pushdown; Avro, CSV and JSON are row formats. ORC is also columnar: the instructor accepts it as partly correct with a justification (best for Hive), but Parquet is the broader standard.

Q3 What does “splittable” mean for a file format in Hadoop?

(a) The file can be deleted in parts by HDFS
(b) Multiple Mappers can process different parts of the file simultaneously
(c) The file format supports splitting schema across multiple NameNodes
(d) The file can be distributed across multiple data centres

> **(b) Multiple Mappers can process different parts of the file simultaneously**
> Reading can start anywhere in the file, which also allows data-local tasks.

Q4 YARN was introduced in which major Hadoop version, and what did it replace?

(a) Hadoop 1.0; replaced the NameNode
(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(c) Hadoop 3.0; replaced DataNodes with Containers
(d) Hadoop 2.0; replaced the Secondary NameNode with a Standby NameNode

> **(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster**
> (d) is the trap: the Standby NameNode is HDFS, not YARN.

True or false (4)

1 Parquet is a row-oriented file format optimised for write-heavy workloads.

Old slides (v1)

> **False.** Parquet is column-oriented and optimised for read-heavy analytics; the row-oriented, append-friendly format is Avro.
>
> **From the lecturer’s files**
>
> **Model justification (old slides):** Parquet is column-oriented and built for read-heavy analytical queries, not for row-by-row writes.

2 YARN allows multiple computation frameworks such as Spark and Tez to share resources on the same Hadoop cluster.

> **True.** Each framework brings its own ApplicationMaster, so YARN runs MapReduce, Spark and Tez on one cluster.

3 Gzip-compressed files are natively splittable in Hadoop, allowing multiple Mappers to process them in parallel.

Old slides (v1)

False. Gzip files are not splittable, so one Mapper reads the whole file; use bzip2 or Parquet + Snappy for parallel reads.

> **From the lecturer's files**
>
> **Model justification (old slides):** Gzip is a streaming format with no internal block boundaries, so it must be decompressed from the start and cannot be split.

4 ORC format is specifically optimised for Hive workloads and achieves among the highest compression ratios of any Hadoop format.

True. ORC is columnar, best for Hive, with the highest compression, Hive ACID and vectorized reads.

Descriptive (3)

D1 Explain YARN's three-component architecture — ResourceManager, NodeManager, and ApplicationMaster. Draw a diagram showing how they interact when a Spark job is submitted to the cluster.

See the diagram on the topic "YARN and its three components"

ResourceManager (1 per cluster): Scheduler grants containers; ApplicationsManager accepts jobs and starts each AM.
NodeManager (1 per node): launches and monitors containers, heartbeats to the RM, enforces CPU/RAM limits.
ApplicationMaster (1 per job): requests containers, has NMs launch tasks, retries failures, releases containers.

Diagram (see the diagram in the YARN and its three components card): Client, RM, Spark AM, NM1–NM3, with numbered arrows:

1. Client submits the job to the RM.
2. RM launches the Spark AM in a container.
3. AM registers with the RM and requests containers; the RM Scheduler grants them.
4. AM tells NM1–NM3 to launch executors.
5. Executors report status to the AM.
6. AM unregisters from the RM when done.

Example: the Spark AM asks for 100 containers {4 cores, 8 GB}; the busy RM grants 80 and the AM runs executors in them.

D2 What is columnar storage? Why does it improve performance for analytics queries that select only a subset of columns? Use a concrete example with numbers to illustrate the I/O reduction.

See the diagram on the topic "Row vs columnar storage and Parquet"

Columnar storage keeps all values of one column together on disk (Parquet, ORC); a row store (CSV, Avro) writes whole records one after another.

Why faster: the query reads only the columns it names; Parquet's min/max stats skip whole row groups (predicate pushdown); same-type values compress better.

Example: 100 columns, 10 TB, query on 3 columns.

Format	Data read
CSV	10 TB
Parquet	10 × 3/100 = 0.3 TB = 300 GB

I/O cut 97%, about **33×** faster.

D3 Compare Avro and Parquet. Explain what "schema evolution" means and why Avro handles it better. In which scenarios would you use each?

Avro	Parquet

Layout	row	columnar
Strength	schema evolution, appends	analytic reads, pushdown
Codec	Snappy/Deflate	Snappy/Gzip

Schema evolution: the schema changes over time (e.g. a new field) while old files and readers keep working.

Why Avro handles it: the writer schema is stored with the data, the reader supplies its own schema, fields match by name; missing fields get defaults, unknown fields are ignored. Backward and forward compatible.

Use: Avro for Kafka → HDFS ingest and data exchange; Parquet for Spark, Presto and ML analytics.

Analytical (2)

A1 Data Lake Performance Migration: A Dhaka-based telecom company stores 10 TB of daily call-detail records (CDR) on HDFS as gzip-compressed CSV files. Their data engineers report that a Hive query selecting 4 of 80 columns takes 6 hours. – Identify two specific reasons why this configuration is slow – Propose a migration strategy: new format, compression codec, and partitioning scheme – Estimate the expected I/O reduction (show your calculation) – What YARN scheduler would you use and why, given that multiple teams share the cluster?

★ Very likely | Lecturer's plan | CLO3

i) Why slow

1. Gzip is not splittable: one Mapper per file, the cluster idles.
2. CSV is a row store: reading 4 columns parses all 80; no stats to skip data.

ii) Migration: ORC (Hive) or Parquet (if Spark reads it too); Snappy for hot data (Zlib for ORC in Hive, Zstd for archive); partition by date and region, e.g. `dt=YYYY-MM-DD/region=`; convert once, then nightly.

iii) I/O estimate

Fraction read	4/80 = 0.05
Data read	10 TB × 0.05 = 500 GB
Reduction	**95%** (20× less)
Time if I/O-bound	360 min × 0.05 ≈ **18 min**

iv) Scheduler: Capacity: guaranteed queues per team (ETL, analysts, data science), idle capacity borrowed, so no team starves another. FIFO lets one long job block all.

> **From the lecturer's files**
>
> The lecturer's plan seeds this as 10 TB of web click data stored as gzip CSV; the same answer applies.

A2 YARN Failure Trace: A NodeManager (NM2) hosting 5 running containers for a MapReduce job fails mid-execution. Trace the complete failure recovery sequence: which components detect the failure, in what order, what state changes occur in the ResourceManager, and how are the 5 containers' tasks restarted.

1. NM2 crashes; its 5 containers die; heartbeats stop.
2. **RM** detects the missed heartbeats after the timeout (>10 min) and declares NM2 lost.
3. **RM state:** NM2 removed from schedulable capacity; its 5 containers marked **failed**.
4. RM notifies the MapReduce **AM**.
5. AM requests 5 replacement containers.
6. RM grants them on healthy nodes (e.g. NM1, NM3), preferring nodes with HDFS replicas of the input.
7. AM has those NMs relaunch the 5 tasks from their input splits; finished maps whose output was on NM2's disk may rerun.

If the AM was on NM2, the RM restarts it (default 2 attempts); MapReduce recovers state from HDFS checkpoints.

Discussion (5)

L1 Resource consolidation increases utilisation: merging 3 siloed clusters into one raised average utilisation from 45% to 78% — the same capacity served 73% more workload

> **Claim:** one shared cluster uses hardware better than three silos.
>
> **For:** LinkedIn's silos (MapReduce, Spark, Samza) were idle at different times; merged, utilisation rose 45% → 78%, and 78/45 = 1.73, so 73% more work on the same hardware. No cross-cluster copying, one config instead of three.
>
> **Against:** workloads now compete, so queues, pre-emption and RM high availability become essential.

L2 Capacity Scheduler enables SLAs: production queues guaranteed minimum resources; analyst adhoc queries cannot starve revenue-generating pipelines

> **Claim:** guaranteed queues protect SLAs (promised finish times).
>
> **For:** production holds 40% at LinkedIn, so the 10% adhoc queue cannot delay revenue pipelines, while idle capacity is still lent out.
>
> **Against:** fixed percentages need tuning, and analysts may wait at busy times; Fair gives short jobs faster turnaround.

L3 Pre-emption needs tuning: aggressive pre-emption killed long-running Spark jobs; LinkedIn set 30-second grace periods and added Spark checkpoint-on-preempt hooks

> **Claim:** pre-emption is needed but must match the workload.
>
> **For:** without reclaiming borrowed containers, queue guarantees are empty promises.
>
> **Against:** killing a container wastes its work; long Spark jobs restarted repeatedly. LinkedIn's fix: a 30-second grace period and checkpoint-on-preempt hooks so jobs resume.

L4 YARN is the OS; frameworks are processes: the paradigm shift — the cluster is the computer, YARN is its kernel

> **Claim:** YARN is to a cluster what an OS kernel is to a computer.
>
> **For:** the RM allocates CPU and RAM as containers, NMs isolate them with cgroups, and frameworks are just applications with their own AM; the same three-tier design lives on in Kubernetes.
>
> **Against:** YARN sizes containers only by CPU and RAM, relies on HDFS for storage, and Kubernetes has replaced it for most new deployments.

L5 Samza on YARN: stream processing is just another set of long-running containers; YARN's fault tolerance restarts failed Samza tasks

> **Claim:** a stream job is just long-running containers.
>
> **For:** Samza shares the cluster in its 15% queue, no separate stream cluster, and YARN restarts failed tasks on other nodes.
>
> **Against:** the containers hold capacity permanently, and the >10 min NodeManager timeout is slow for real-time data, so Samza must resume from the right Kafka offset itself.

Live exercises (1)

LE Task (20 minutes, pairs) Model LinkedIn's 5,000-node YARN cluster in Python. Given: • 5,000 nodes; each: 64 GB RAM, 16 vCPUs • Capacity Scheduler: production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10% • Container sizes: Hive mapper: {4 GB, 2 vCPU}; Spark executor: {8 GB, 4 vCPU}; Samza task: {2 GB, 1 vCPU} Questions: 1 Max concurrent containers per queue? 2 Which resource is the bottleneck (CPU or RAM)? 3 If production needs 10,000 Hive mappers, how many nodes are required?

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4, "cpu": 2},
  "spark":      {"ram": 8, "cpu": 4},
  "samza":      {"ram": 2, "cpu": 1},
  "adhoc":      {"ram": 4, "cpu": 2},
}

for q, frac in queues.items():
    nodes_q = NODES * frac
    ram_q    = nodes_q * RAM_GB
    cpu_q    = nodes_q * VCPUS
    if q in containers:
        c = containers[q]
        by_ram = ram_q // c["ram"]
        by_cpu = cpu_q // c["cpu"]
        max_c = int(min(by_ram, by_cpu))
        bottleneck = ("RAM" if by_ram < by_cpu
                            else "CPU")
        print(f"{q}: {max_c} containers "
              f"(bottleneck: {bottleneck})")
```

Task: run the skeleton; add Q3:

```
import math
c = containers["production"]
per_node = min(RAM_GB // c["ram"], VCPUS // c["cpu"])
print(per_node, math.ceil(10_000 / per_node))
```

Expected output:

```
production: 16000 containers (bottleneck: CPU)
spark: 5000 containers (bottleneck: CPU)
samza: 12000 containers (bottleneck: CPU)
adhoc: 4000 containers (bottleneck: CPU)
8 1250
```

1. production 16,000; spark 5,000; samza 12,000; adhoc 4,000; reserve has no container size, so none.

2. CPU. A node has 64/16 = 4 GB per vCPU; every container asks 2 GB per vCPU, so vCPUs run out with half the RAM idle.

3. Per node min(64/4, 16/2) = 8 mappers; 10,000 / 8 = **1,250 nodes**, which fits in production's 2,000.

Topics

YARN and its three components

In one line

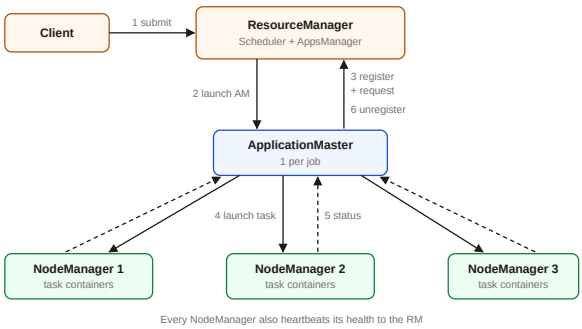
YARN is Hadoop 2's resource manager: one ResourceManager gives out containers, a NodeManager runs them on each node, and one ApplicationMaster manages each job.

Traps

- NameNode and DataNode are HDFS (storage). They are not YARN parts and do not allocate CPU or RAM.
- The AM is per job, the RM is per cluster. Q1's wrong favourite is the AM.
- Hadoop 2.0 also added the Standby NameNode. That is HDFS high availability, not YARN (Q4 distractor).
- The AM requests containers; it does not grant them. Only the RM's Scheduler grants.

Spot it in a scenario

If the question describes...	Answer
one process decides which application gets resources across the whole cluster	**ResourceManager (its Scheduler)**
an agent on each machine starts tasks and reports health every few seconds	**NodeManager**
a Spark job asks for 100 executors and gets 80	**ApplicationMaster requests, RM Scheduler grants**
a company wants Spark and MapReduce on one cluster but runs Hadoop 1.x	**move to YARN (JobTracker is MapReduce-only)**
a bundle of 4 vcores and 8 GB on one node	**container**



A job on YARN: the client submits to the ResourceManager, which starts the job's ApplicationMaster; the AM requests containers and has NodeManagers launch tasks, which report status back.

YARN fault tolerance

In one line

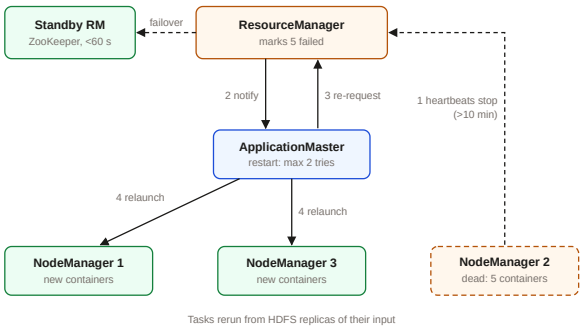
When a NodeManager stops heartbeating, the RM marks its containers failed and the AM requests replacements on other nodes.

Traps

- The AM, not the RM, re-requests the replacement containers. The RM only detects and marks.
- The NM timeout is over 10 minutes, not seconds. The 60 seconds figure is RM failover.
- Recovery from checkpoint depends on the framework: MapReduce can, early Spark could not.

Spot it in a scenario

If the question describes...	Answer
a worker machine stops reporting and its tasks reappear on other machines	**NodeManager failure, heartbeat timeout, AM re-requests containers**
the job's coordinator crashes and the RM starts a new one, within a set limit of attempts	**AM failure, restarted up to am.max-attempts (default 2)**
the master node dies and a backup takes over within a minute	**RM High Availability via ZooKeeper**
a stream processor runs in containers that never finish and is restarted on failure	**Samza on YARN (long-running containers)**



NodeManager 2 dies: the RM notices missed heartbeats, marks its containers failed and tells the AM, which re-requests and relaunches the tasks on healthy nodes; a Standby RM covers RM failure.

Schedulers and capacity planning

In one line

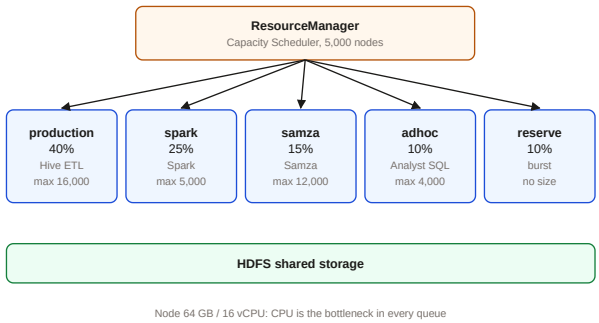
The Capacity Scheduler splits the cluster into queues with guaranteed shares and lets queues borrow idle capacity.

Traps

- Take the **minimum** of the by-RAM and by-CPU counts, never the RAM count alone.
- The reserve queue has no container size, so it gets no count.
- Capacity is not a hard cap: idle capacity is lent out and reclaimed by pre-emption.
- 73% is 78/45 – 1, not 78 – 45 = 33 points.

Spot it in a scenario

If the question describes...	Answer
several teams share a cluster and production must always get at least 40%	**Capacity Scheduler**
a single huge job submitted first makes everyone else wait	**FIFO weakness**
many short interactive queries should finish fast even while a big job runs	**Fair Scheduler**
jobs differ in their CPU and RAM needs and fairness must cover both	**DRF**
a queue gets its lent capacity back by killing containers after a warning	**pre-emption with a grace period**



The live exercise's Capacity Scheduler queues on LinkedIn's 5,000-node cluster, with each queue's maximum containers. (The case study's real split had tez 7% and reserve 3%; the exercise merges them into reserve 10%.)

Worked example

Live exercise (P2 slide 23): 5,000 nodes, each 64 GB and 16 vCPU. Queues production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10%. Containers: Hive mapper {4 GB, 2 vCPU}, Spark executor {8 GB, 4 vCPU}, Samza task {2 GB, 1 vCPU}; adhoc uses the Hive mapper size.

Queue	Nodes	RAM GB	vCPU	By RAM	By CPU	Max
production	2,000	128,000	32,000	32,000	16,000	**16,000**
spark	1,250	80,000	20,000	10,000	5,000	**5,000**
samza	750	48,000	12,000	24,000	12,000	**12,000**
adhoc	500	32,000	8,000	8,000	4,000	**4,000**

Bottleneck: **CPU** everywhere. Nodes for 10,000 Hive mappers: **1,250**.

Step by step

1. Given 5,000 nodes and spark share 25% -> spark nodes = 5,000 × 0.25 = 1,250.
2. Given 1,250 nodes of 64 GB -> RAM = 80,000 GB; of 16 vCPU -> vCPU = 20,000.
3. Given 80,000 GB and 8 GB executors -> by RAM = 10,000.
4. Given 20,000 vCPU and 4-vCPU executors -> by CPU = 5,000.
5. Given by RAM 10,000 and by CPU 5,000 -> max = 5,000, bottleneck CPU.
6. Given a node of 64 GB / 16 vCPU and a mapper of 4 GB / 2 vCPU -> per node = min(16, 8) = 8.
7. Given 8 mappers per node and 10,000 mappers -> nodes = 10,000 / 8 = 1,250 (production has 2,000, so it fits).

Variants

- Quick bottleneck test: node RAM per vCPU = 64/16 = 4 GB; every container asks 2 GB per vCPU, so CPU runs out first with half the RAM idle.
- 2,000 nodes of 128 GB / 32 vCPU, spark 25%: 500 nodes, 64,000 GB, 16,000 vCPU; by RAM 8,000, by CPU 4,000; max 4,000 (CPU).
- 3,000 Spark executors {8 GB, 4 vCPU} on 64 GB / 16 vCPU nodes: per node min(8, 4) = 4; nodes = 750.
- Utilisation 45% to 78%: 78 / 45 = 1.73, so 73% more work on the same hardware.

Row vs columnar storage and Parquet

In one line

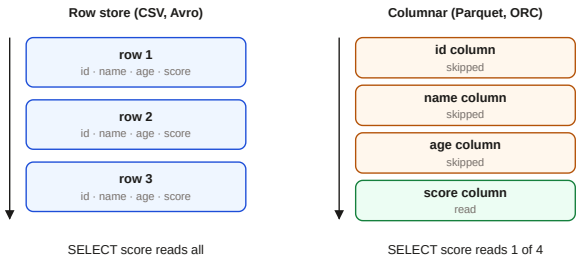
Columnar storage keeps each column's values together on disk, so a query reads only the columns it needs.

Traps

- Avro is a row format even though it is binary and self-describing.
- The 33× assumes columns of equal size and an I/O-bound query; it is an estimate.
- Skip a row group only when its range cannot match: for `> 90` look at the max, for `< 70` look at the min.
- Pushdown is per row group; the whole file is not skipped unless every row group fails.

Spot it in a scenario

If the question describes...	Answer
a query needs 3 of 100 columns and the files are CSV	**switch to a columnar format**
a filter skips whole blocks using stored min and max values	**predicate pushdown**
a footer lists stats for each chunk of each column	**Parquet footer / column chunk statistics**
a 10 TB table is queried but only 50 GB is read from disk	**columnar layout plus predicate pushdown**



A row store keeps whole records together, so reading one column reads them all; a columnar store keeps each column together and skips the rest.

Worked example

Slide (P1 slides 6, 16): D = 10 TB, N = 100 columns, k = 3 read.

- Read = D × k / N = 10 TB × 3/100 = 0.3 TB = **300 GB**
- I/O reduction = 1 – 3/100 = **97%**
- Speedup ≈ N / k = 100/3 ≈ **33×**

Pushdown (slide 23): 10 TB on disk, 50 GB read: 50/10,000 = 0.005, so 99.5% less, 200× faster.

Step by step

1. Given k = 4 of N = 80 columns -> fraction read = 4/80 = 0.05.
2. Given fraction 0.05 and D = 10 TB -> read = 0.5 TB = 500 GB.
3. Given fraction 0.05 -> reduction = 1 – 0.05 = 95%; speedup = 80/4 = 20×.
4. Given 6 h I/O-bound and fraction 0.05 -> new time ≈ 360 × 0.05 = 18 min.
5. Given row group min/max [60, 85] and filter score > 90 -> max 85 < 90, skip.

Variants

- D = 20 TB, N = 200, k = 5: read 0.5 TB, reduction 97.5%, speedup 40×.
- 2 TB on disk, 40 GB read after pushdown: 40/2,000 = 0.02, reduction 98%, speedup 50×.
- Filter score < 70 with RG1 [60, 85], RG2 [85, 99]: read RG1 (min 60 < 70), skip RG2 (min 85).

File formats: Avro, Parquet, ORC

In one line

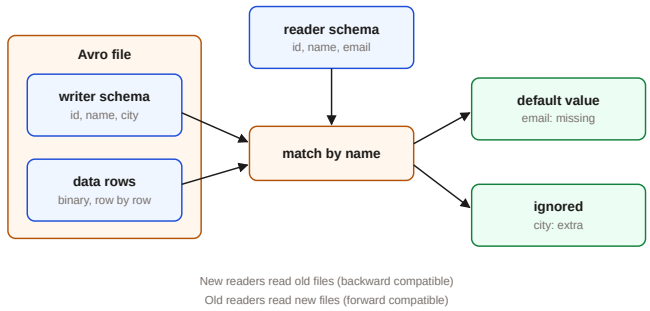
Avro is a row format built for schema evolution; Parquet and ORC are columnar formats built for analytics.

Traps

- Avro is self-describing and binary, but it is row-oriented, not columnar.
- ORC is also correct for analytics on Hive; Parquet is the broader standard (Q2 note).
- Missing field = default value; extra field = ignored. Do not swap them.
- JSON lines is not splittable in raw form.

Spot it in a scenario

If the question describes...	Answer
event producers keep adding fields and old consumers must not break	**Avro schema evolution**
a Spark team runs aggregations over a few columns of billions of rows	**Parquet + Snappy**
a Hive warehouse needs the smallest files and ACID updates	**ORC + Zlib**
data older than 2 years must be stored as cheaply as possible	**ORC or Parquet + Zstd**
an ingest layer lands Kafka events, converted nightly for analytics	**Avro at ingest, Parquet for analytics**



Avro schema evolution: the writer schema travels in the file, the reader brings its own, fields are matched by name, missing ones get defaults and extra ones are ignored.

Compression codecs and splittability

In one line

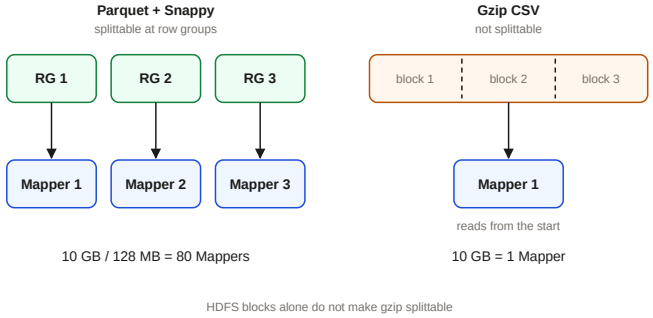
A splittable file can be read from any point, so multiple Mappers process different parts at the same time.

Traps

- HDFS splitting a gzip file into blocks does not make it splittable for MapReduce.
- Snappy alone is not splittable; Parquet + Snappy is, at row-group boundaries.
- Bzip2 is splittable, gzip is not; T/F 3 hinges on this.
- Round splits up: a partial last block still needs a Mapper.

Spot it in a scenario

If the question describes...	Answer
a 50 GB compressed CSV keeps one Mapper busy for hours while the cluster idles	**gzip is not splittable**
three Mappers read three row groups of one file at once	**splittable Parquet**
data must stay as plain text but still be split	**bzip2 (or LZO with an index)**
hot analytics data must decompress quickly	**Snappy inside Parquet**



Parquet row groups each feed their own Mapper; a gzip CSV is one stream, so one Mapper reads it all even though HDFS stores it in blocks.

Worked example

Slide (P1 slides 20-21): a 10 GB gzip CSV gets **1 Mapper**. The same 10 GB as Parquet with 128 MB row groups gets 10 × 1,024 / 128 = **80** splits, so up to 80 Mappers in parallel.

Rule: splits = ⌈F / B⌉ if splittable, 1 if not.

Step by step

1. Given a gzip file of any size -> splits = 1.
2. Given F = 10 GB -> F = 10 × 1,024 = 10,240 MB.
3. Given 10,240 MB and B = 128 MB -> splits = 10,240 / 128 = 80.
4. Given 80 splits -> up to 80 Mappers in parallel.

Variants

- 10 GB with 256 MB row groups: 10,240 / 256 = 40 Mappers.
- 1 GB with 128 MB row groups: 1,024 / 128 = 8 Mappers.
- 300 MB with 128 MB blocks: ⌈300 / 128⌉ = ⌈2.34⌉ = 3 Mappers.

Week 6

Apache Spark: RDDs and in-memory computing

Spark keeps working data in RAM so iterative jobs stop paying MapReduce's disk cost on every pass. The chapter covers how Spark is built, how RDDs recover from failures through lineage, lazy evaluation, the narrow/wide split that decides shuffles and stages, and DataFrames.

Exam questions

Multiple choice (4)

Q1 What makes Spark significantly faster than MapReduce for iterative algorithms?

(a) Spark uses more CPU cores per node
(b) Spark writes intermediate results to faster SSDs
(c) Spark caches intermediate data as RDDs in memory
(d) Spark skips the shuffle phase entirely

★ Very likely | Lecturer's plan | CLO3

> **(c) Spark caches intermediate data as RDDs in memory**
>
> Later iterations read from RAM instead of re-reading and re-writing HDFS; the hardware is the same.

Q2 How does Spark achieve fault tolerance in RDDs without full data replication?

(a) Checkpointing to HDFS every 60 seconds
(b) Tracking lineage and recomputing lost partitions from parent RDDs
(c) Copying each RDD partition twice across two Executors
(d) Using a write-ahead log on the Driver node

★ Very likely | Lecturer's plan | CLO3

> **(b) Tracking lineage and recomputing lost partitions from parent RDDs**
>
> The recipe is kept, not copies; (c) is replication.

Q3 Which of the following is **NOT** a component of the Spark ecosystem?

(a) MLlib
(b) GraphX
(c) Structured Streaming
(d) HBase

★ Very likely | Lecturer's plan | CLO3

> **(d) HBase**
>
> HBase is a separate NoSQL database on HDFS; Spark can only read it as a data source.

Q4 In Spark, a “transformation” such as map() or filter() is:

(a) Immediately executed when called and materialised to memory
(b) Lazily evaluated — only executed when a subsequent action is called
(c) Automatically persisted to HDFS after execution
(d) Only available in the Scala API, not in PySpark

★ Very likely | Lecturer's plan | CLO3

> **(b) Lazily evaluated — only executed when a subsequent action is called**
>
> `rdd.filter(f)` alone does nothing; `rdd.filter(f).count()` runs it.

True or false (4)

1 Spark can only run on top of Hadoop YARN and has no other deployment option.

★ Very likely | Lecturer's plan | Old slides (v1) | CLO3

False. The Cluster Manager can be YARN, Kubernetes, Mesos or Spark Standalone.

> **From the lecturer's files**
>
> **Model justification (old slides):** Spark runs on YARN, Kubernetes, Apache Mesos or its own Standalone manager.

2 RDDs are mutable — individual elements can be modified in place after creation.

★ Very likely | Lecturer's plan | Old slides (v1) | CLO3

False. RDDs are immutable; every transformation creates a new RDD.

> **From the lecturer's files**
>
> **Model justification (old slides):** RDDs are immutable; a transformation creates a new RDD and never changes the original.

3 Spark's DataFrame API provides schema-aware, named-column operations similar to SQL tables.

★ Very likely | Lecturer's plan | CLO3

True. A DataFrame has named columns and a schema, like a relational table, and accepts SQL.

4 MapReduce is generally better than Spark for machine learning workloads that require many iterations over the same dataset.

★ Very likely | Lecturer's plan | Old slides (v1) | CLO3

False. Spark is better: it caches the data in RAM and is 10–100× faster on iterative work, while MapReduce hits disk every iteration.

> **From the lecturer's files**
>
> **Model justification (old slides):** For iterative ML Spark is 10–100× faster because it caches the training data in memory; MapReduce re-reads HDFS on every iteration.

Descriptive (3)

D1 Explain the concept of a **Resilient Distributed Dataset (RDD)**. What does *"resilient"* mean in this context? How does Spark recover a lost RDD partition without replication?

RDD: a read-only, partitioned collection of records, created from stable storage (HDFS, S3) or by transforming another RDD. Properties: immutable, partitioned, lazy, lineage, resilient.

Resilient: it survives a node failure by recomputing lost partitions from its lineage (the steps that built it) instead of keeping 3 copies. The computation is the backup.

Recovery:

1. Spark detects Part 2 of mappedRDD is lost.
2. Traces lineage: mappedRDD ← filteredRDD ← inputRDD.
3. Re-reads input Part 2 from HDFS.
4. Re-applies filter, then map.
5. Part 2 restored; the job is not restarted.

Diagram: draw the lineage chain shown in the diagram in the RDD card: inputRDD → filteredRDD → mappedRDD, with Part 2 of mappedRDD crossed out and the recovery path going back to inputRDD Part 2 on HDFS.

Cheap for narrow dependencies; checkpoint long chains.

D2 Differentiate between **Spark transformations** and **actions**. Give two concrete examples of each. Why does Spark not execute transformations immediately?

	Transformation	Action
Runs	Lazily, only recorded in the DAG	Immediately, triggers the DAG
Returns	A new RDD	A value to the Driver, or output to storage
Examples	`map(f)`, `filter(f)`	`count()`, `collect()`

Why wait: with the whole DAG in view, Spark can

- optimise the plan (DAG scheduler, or Catalyst for DataFrames);
- pipeline filter and map in one pass, with no intermediate disk writes;
- run only the partitions needed, e.g. `first()` stops at the first match.

D3 Describe Spark's **lazy evaluation model**. What is the advantage of deferring computation until an action is called? Use the example `rdd.filter(x > 1000).first()` to illustrate.

See the diagram on the topic “Transformations, actions and lazy evaluation”

Model: transformations are only recorded in the DAG. When an action is called, Spark analyses the DAG, optimises it, pipelines compatible steps in one pass per partition, and runs only the partitions it needs.

Advantages: less work (early stop, skipped partitions), no intermediate disk writes, whole-plan optimisation.

Example:

1. `filter(x > 1000)`: recorded, no data read.
2. `first()`: action; Spark starts reading the first partition.
3. It returns the first value above 1000 and stops.
4. Eager execution would filter the entire dataset, then keep one value.

Analytical (2)

A1 **Iterative I/O Cost Analysis:** You need to train a K-Means clustering model over **50 GB** of bKash transaction data with **100 iterations**. Assume HDFS read speed = 500 MB/s and RAM cache read speed = 5,000 MB/s.

- Calculate the total HDFS I/O for MapReduce (all 100 iterations)
- Calculate the total I/O for Spark (50 GB fits in cluster RAM)
- Estimate the wall-clock time for each approach
- Under what conditions would Spark NOT outperform MapReduce?

★ Very likely | Lecturer's plan | CLO3

Given: D = 50 GB = 50,000 MB, k = 100, HDFS 500 MB/s (write assumed = read), RAM 5,000 MB/s.

	MapReduce	Spark
HDFS I/O	100 × (50 + 50) = **10 TB**	**50 GB** (read once); RAM reads 99 × 50 = 4,950 GB
Per pass	50,000/500 = 100 s read + 100 s write = 200 s	100 s first, then 50,000/5,000 = 10 s
Total time	100 × 200 = **20,000 s ≈ 5.56 h**	100 + 99 × 10 = **1,090 s ≈ 18.2 min**

Disk I/O 200× less; time speedup 20,000 / 1,090 ≈ **18.3×**.

Spark does not win when: the job is single-pass (nothing to reuse); the data does not fit in RAM (spill or evict and recompute); Executor memory (~8 GB minimum) is too costly; stable MapReduce code is already fast enough.

From the lecturer’s files

> The lecturer's plan words this with 100 ms per 128 MB block instead of MB/s; see the worked version “K-Means, 50 GB, 100 iterations at 100 ms per 128 MB block” in Practise.

A2 **RDD Failure Recovery Trace:** A Spark job computes `input` → `filteredRDD` → `mappedRDD` → `reducedRDD` over 10 partitions. Mid-execution, the Executor holding partitions 3 and 7 of `reducedRDD` crashes. Trace the complete recovery sequence: what does the Driver detect, which partitions of which RDDs need to be recomputed, and what is the worst-case recovery cost if `reducedRDD` involves a **wide** dependency (`groupByKey`)?

★ Very likely | Lecturer's plan | CLO3

Stages: filter, map narrow → Stage 1 (input → filteredRDD → mappedRDD). groupByKey wide → Stage 2 (reducedRDD).

Driver detects: the Executor is lost; tasks for reducedRDD partitions 3 and 7 failed. The other 8 partitions are fine.

Lineage: reducedRDD ← groupByKey ← mappedRDD ← map ← filteredRDD ← filter ← input.

Case	Recomputed
If all narrow	Partitions 3 and 7 at each level: 2 × 4 = 8 computations (20%)
Wide, shuffle files survive	Only reducedRDD 3 and 7, re-fetching from all 10 mappedRDD outputs
Wide, worst case	Shuffle output lost too: all 10 input, filteredRDD and mappedRDD partitions (30) + reshuffle + reducedRDD 3, 7 (2) ≈ the whole job

Limit the cost: reduceByKey instead of groupByKey, cache mappedRDD, checkpoint long chains.

> **From the lecturer’s files**
>
> The lecturer's plan pairs this with the K-Means question: executor crash recovery via lineage, and its worst-case cost.

Discussion (5)

1 **In-memory is a multiplier, not a panacea**: Spark is 10–100× faster on iterative workloads; for single-pass ETL over data larger than RAM, gains are smaller (2–5×)

Claim: memory speeds up reuse; it does not fix every workload.

For: iterative jobs reuse the working set. RDD paper: logistic regression 127 s vs 0.96 s per iteration (132×).

Against: single-pass ETL reads each byte once, and data larger than RAM spills to disk, so the gain falls to 2–5×.

Choose by workload: more than 2 iterations favours Spark.

2 **The lineage model scales elegantly**: re-computing a lost partition from lineage is cheaper than maintaining 3× replicated in-memory state

Claim: lineage beats replication for in-memory data.

For: 3× replication triples RAM and network writes on every step; lineage costs nothing until a failure and then rebuilds only lost partitions.

Against: with wide dependencies or long chains, one lost partition can need many ancestors recomputed, so checkpoints are still needed.

3 **Shuffle is the bottleneck**: both at GraySort and in production; minimising wide dependencies (shuffle) is the primary Spark tuning skill

Claim: shuffles dominate Spark's cost.

For: a shuffle moves data over the network, writes to disk and forces a stage barrier. Spark's GraySort win came from a cheaper shuffle: sort-based shuffle (about 120 TB less I/O), all 8 SSDs as shuffle directories, one sortByKey. Tuning: reduceByKey over groupByKey, co-partition before joins.

Against: sorts and un-partitioned joins need a shuffle; you minimise it, you cannot remove it.

---

4 **Benchmark marketing is real**: GraySort made Spark famous; it also hid cases where Spark's JVM memory overhead hurt performance (many small files, high-cardinality `groupBy`)

Claim: a headline benchmark sells a system but shows one workload.

For: 100 TB in 23 min on 206 nodes vs Hadoop's 72 min on 2,100 drove Spark's adoption.

Against: it ran on tuned large nodes (244 GB RAM, 8 SSDs). Many small files (per-task overhead) and high-cardinality groupBy (too many keys for JVM memory, GC pauses) were not tested.

Test on your own workload.

---

5 **Open-source velocity**: the Apache Spark community grew to 1,000+ contributors in 5 years; Databricks' business model (cloud + support) aligned commercial incentives with open-source development

Claim: a free engine plus a paid service sped up Spark.

For: Databricks earns from cloud and support, not licences, so improving open Spark helps its business. 1,000+ contributors in 5 years produced Spark SQL, MLlib, Structured Streaming, GraphX.

Against: one company steering the project can shape the roadmap and push features to its paid product first (lock-in risk).

Live exercises (4)

Task **Task (25 minutes, pairs)**
Run the following PySpark script locally (Spark local mode).
Goal: build and inspect an RDD lineage; observe lazy evaluation; compare cached vs. uncached timing.

```
from pyspark import SparkContext
import time
sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")
# Build lineage (lazy - no compute yet)
lines = sc.textFile("README.md")
words = lines.flatMap(lambda l: l.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a+b)
# Inspect lineage BEFORE action
print(counts.toDebugString().decode())
# First action - triggers full DAG
t0 = time.time()
n = counts.count()
print(f"count={n}  time={time.time()-t0:.2f}s")
# Cache and re-run
counts.cache()
counts.count()          # warm up cache
t1 = time.time()
n = counts.count()      # served from cache
print(f"cached count={n} "
      f"time={time.time()-t1:.4f}s")
sc.stop()
```

`local[4]` = local mode, 4 threads. textFile, flatMap, map, reduceByKey only build lineage; the first `count()` runs the DAG. Key additions for the questions (run with `python rdd_demo.py`):

```
print(counts.toDebugString().decode())   # Q1 before cache
for i in (1, 2):                          # Q2 two uncached runs
    t0 = time.time(); counts.count()
```

```
    print(i, time.time() - t0)
counts.cache()
print(counts.toDebugString().decode())   # Q1 after cache
counts.count()                            # fills the cache
t1 = time.time(); counts.count()
print("cached", time.time() - t1)
# Q3: counts = pairs.groupByKey().mapValues(sum)
input("open localhost:4040, then Enter")
```

Expected: the lineage shows HadoopRDD → Python RDDs, with a `+-` indent at the reduceByKey shuffle; count = number of distinct lower-cased words.

1 Call `rdd.toDebugString()` before and after `cache()` — what changes?

The parent chain does not change, because cache() is lazy and lineage is kept for recovery. After cache() the top line shows a storage level (e.g. `[Memory Serialized 1x Replicated]`). After an action fills the cache it also shows `CachedPartitions` and memory size (expected).

2 Time `rdd.count()` twice without `cache()`. Then add `.cache()` and time again. Report the speedup.

Speedup = uncached time / cached time. Expected: run 1 slowest (start-up, read, shuffle); run 2 a bit faster (shuffle files reused, Stage 1 skipped); cached run fastest. On a small README start-up overhead dominates, so the ratio is modest; the RDD paper shows 127 s → 0.96 s (132×) at scale.

3 Change `reduceByKey` to `groupByKey`. Use the Spark UI (localhost:4040) to compare shuffle read/write bytes between the two.

Use `pairs.groupByKey().mapValues(sum)`, then read Shuffle Write/Read on the UI's Stages tab. Expected: same counts, larger shuffle bytes for groupByKey, because reduceByKey combines each partition's pairs before the shuffle and groupByKey sends every `(word, 1)` pair.

Topics

Why Spark: the MapReduce iteration bottleneck

In one line

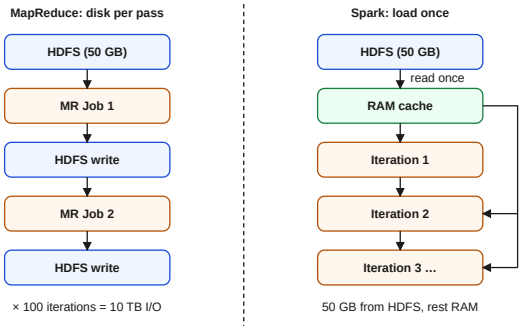
MapReduce writes every intermediate result to HDFS, so iterative algorithms pay disk I/O on every pass; Spark caches the data in RAM once.

Traps

- The 200× is disk I/O, not time. Here RAM is only 10× faster than HDFS, so the time speedup is about 18×.
- Spark iterations 2–100 still read 50 GB each, from RAM. That is 99 RAM passes, not 100.
- If the HDFS write speed is not given, say you assume it equals the read speed.
- Spark does not skip the shuffle. Wide operations like reduceByKey still shuffle.

Spot it in a scenario

If the question describes...	Answer
a model is trained with 50 passes over the same transaction table	**iterative workload: Spark with the data cached in RAM**
a nightly job reads 5 TB of logs once, converts the format and writes it out	**single-pass ETL: Spark gains only 2–5×; MapReduce is acceptable**
analysts want sub-second answers while exploring data in a shell	**Spark interactive queries (MapReduce takes over 30 s)**
the working data is larger than the cluster's total RAM	**Spark spills or evicts and recomputes, so it loses most of its edge**



MapReduce writes to HDFS between every job, while Spark reads the 50 GB once into a RAM cache that feeds every later iteration.

Worked example

K-Means on 50 GB (50,000 MB), 100 iterations. HDFS 500 MB/s (write assumed equal to read), RAM 5,000 MB/s.

	MapReduce	Spark
HDFS I/O	100 × (50 + 50) = 10,000 GB = 10 TB	50 GB (read once)
Time per pass	100 s read + 100 s write = 200 s	first 100 s, then 10 s from RAM
Total time	100 × 200 = 20,000 s ≈ 5.56 h	100 + 99 × 10 = 1,090 s ≈ 18.2 min

Disk I/O ratio 10,000 / 50 = **200×**. Time speedup 20,000 / 1,090 ≈ **18.3×**.

Step by step

1. Given D = 50 GB, k = 100, MapReduce reads and writes D each iteration → MR I/O = 100 × (50 + 50) = 10,000 GB = 10 TB
2. Given Spark reads HDFS only in iteration 1 → Spark HDFS I/O = 50 GB
3. Given 10,000 GB and 50 GB → disk I/O ratio = 10,000 / 50 = 200×
4. Given 50,000 MB at 500 MB/s → one HDFS pass = 100 s; one MR iteration (read + write) = 200 s
5. Given 200 s per iteration → T_MR = 100 × 200 = 20,000 s ≈ 5.56 h
6. Given RAM at 5,000 MB/s → one RAM pass = 10 s; T_Spark = 100 + 99 × 10 = 1,090 s ≈ 18.2 min
7. Given 20,000 s and 1,090 s → speedup = 20,000 / 1,090 ≈ 18.3×

Variants

- D = 20 GB, k = 30, HDFS 400 MB/s, RAM 4,000 MB/s: MR I/O = 30 × 40 = 1,200 GB (60× Spark's 20 GB); T_MR = 30 × 100 = 3,000 s; T_Spark = 50 + 29 × 5 = 195 s; speedup ≈ 15.4×
- Part 2 slide: 10 iterations over 100 GB at 100 MB/s, reads only: 10 × 100,000 / 100 = 10,000 s ≈ 2.8 h (the slide's "≈ 3 hours")
- Per-iteration benchmark (RDD paper): logistic regression 127 s vs 0.96 s → 127 / 0.96 ≈ 132×; K-Means 150 s vs 5 s → 30×

Spark architecture

In one line

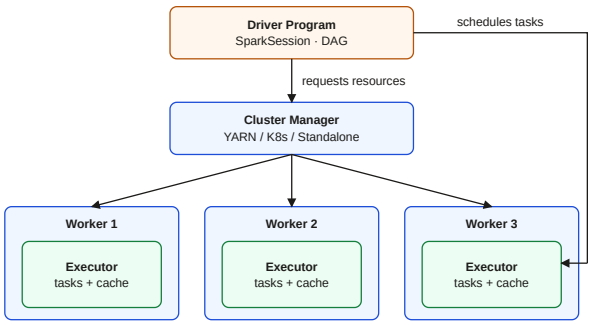
Spark has three tiers: a Driver that builds the DAG and schedules tasks, a Cluster Manager that allocates resources, and Executors that run tasks and cache data.

Traps

- Spark does not need YARN. Kubernetes, Mesos and Standalone also work.
- Cached data lives in the Executors, not in the Driver.
- SparkContext did not disappear in Spark 2; it sits inside SparkSession.

Spot it in a scenario

If the question describes...	Answer
the process that builds the plan and hands out tasks	**Driver**
Spark runs on the company's existing Hadoop resource manager	**Cluster Manager = YARN**
a JVM on a worker machine runs tasks and keeps partitions in RAM	**Executor**
Spark 3 code starts with spark.read and spark.sql	**SparkSession, the unified entry point**



The Driver asks the Cluster Manager for resources, then schedules tasks on Executors that run on Worker Nodes and hold the cache.

RDDs and lineage-based fault tolerance

In one line

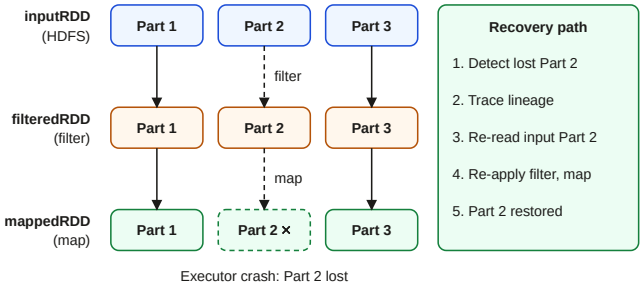
An RDD is a read-only, partitioned collection of records that Spark rebuilds from its lineage if a partition is lost.

Traps

- Checkpointing is an optional add-on for long chains. Lineage is the main mechanism.
- Copying each partition to two Executors is replication, which RDDs avoid.
- `cache()` is lazy. Nothing is stored until an action runs.
- Resilient means recomputable, not replicated.

Spot it in a scenario

If the question describes...	Answer
a worker dies and Spark rebuilds only the missing piece by re-running filter and map	**lineage-based recovery**
an iterative job's chain of steps grows very long and recovery gets slow	**checkpoint the RDD to HDFS**
every iteration of a training loop reads the same lookup data	**cache()/persist() it (MEMORY_ONLY)**
a student tries to change one element of an RDD in place	**not possible: RDDs are immutable; map to a new RDD**



When Part 2 of mappedRDD is lost, Spark follows the lineage back to inputRDD and re-runs filter and map for that one partition.

Transformations, actions and lazy evaluation

In one line

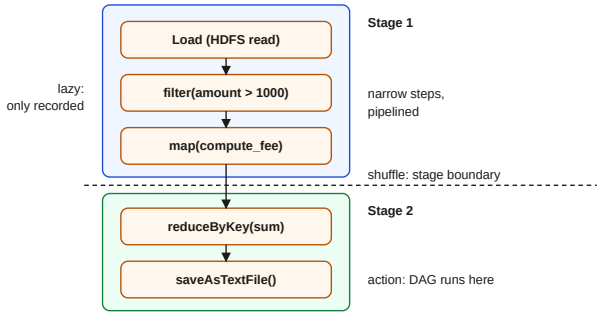
Lazy evaluation means Spark records transformations in a DAG and runs nothing until an action is called.

Traps

- cache() is a transformation (lazy), not an action.
- reduceByKey is a transformation; reduce is an action.
- map and filter exist in PySpark, not only in Scala.
- A transformation's result is not written to HDFS automatically.

Spot it in a scenario

If the question describes...	Answer
the script runs instantly until count() is added, then takes minutes	**lazy evaluation: the action triggered the work**
only one sample transaction above 50,000 BDT is needed	**first() or take(1): Spark stops early**
the result must be saved to HDFS	**saveAsTextFile, an action**
filter and map run in one pass over each partition with no disk writes	**pipelining within a stage**



Transformations are only recorded; the action saveAsTextFile runs the DAG, with narrow steps pipelined in Stage 1 and the reduceByKey shuffle starting Stage 2.

Narrow vs wide dependencies, stages and shuffle

In one line

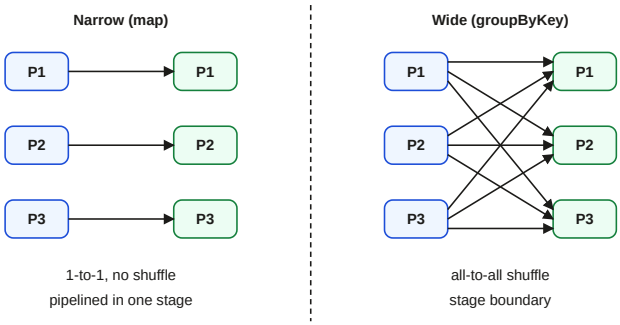
A wide dependency is one where an output partition needs data from many input partitions, which forces a shuffle and a new stage.

Traps

- reduceByKey is still wide. It shuffles, only less data.
- coalesce is narrow (no shuffle); repartition is wide.
- join is narrow only when both RDDs are co-partitioned on the key.
- Part 2 slide 2 says “3.5× fewer machines”, but 2,100 / 206 ≈ 10.2×, as slide 14 says.

Spot it in a scenario

If the question describes...	Answer
word counts are slow and the Spark UI shows huge shuffle bytes from groupByKey	**switch to reduceByKey (local combine)**
two RDDs are partitioned on customer_id and joined with no shuffle	**co-partitioning makes the join narrow**
filter then map run back to back in one stage	**narrow dependencies, pipelined**
losing one partition after a groupByKey forces many parent partitions to rerun	**wide dependency recovery is expensive**



In a narrow dependency each child partition reads one parent partition; in a wide one every child reads from every parent, which forces a shuffle and a new stage.

Worked example

A2: input → filteredRDD → mappedRDD → reducedRDD (groupByKey), 10 partitions. The Executor with reducedRDD partitions 3 and 7 crashes.

Case	Work redone
All narrow (comparison)	Partitions 3 and 7 only: 2 / 10 = 20%; 2 × 4 = 8 partition computations
Wide, shuffle files survive	Only the 2 Stage-2 tasks, re-fetching from all 10 map outputs
Wide, worst case (shuffle files lost)	Stage 1 again: 10 input + 10 filter + 10 map = 30, plus 2 reduce tasks

Shuffle size: 4 partitions × 1,000 pairs, 50 distinct words each. groupByKey 4,000 records; reduceByKey ≤ 4 × 50 = 200, 95% less.

Step by step

1. Given filter and map are narrow and groupByKey is wide → Stage 1 = input → filter → map; Stage 2 = groupByKey
2. Given partitions 3 and 7 lost with only narrow steps → redo 2 / 10 = 20% of the work
3. Given groupByKey → each lost partition needs data from all 10 mappedRDD partitions
4. Given shuffle files lost → rerun Stage 1: 10 + 10 + 10 = 30 partition computations, then 2 reduce tasks
5. Given 4 partitions × 1,000 pairs → groupByKey shuffles 4,000 records
6. Given 50 distinct words per partition → reduceByKey shuffles at most 4 × 50 = 200, i.e. 1 – 200/4,000 = 95% less
7. Given numPartitions = 4 and a key with hash 17 → 17 % 4 = 1, so partition 1

Variants

- Hashes 42, 10, 23, 31 with numPartitions = 4 → partitions 2, 2, 3, 3
- 3 of 12 partitions lost, all narrow → 3 / 12 = 25% of the work redone
- 8 partitions × 500 pairs, 20 distinct keys each → groupByKey 4,000; reduceByKey ≤ 160; 96% less
- GraySort ratios: 72 / 23 ≈ 3.1× faster; 2,100 / 206 ≈ 10.2× fewer nodes; (72 × 2,100) / (23 × 206) = 151,200 / 4,738 ≈ 32× better efficiency per node

DataFrames, Spark SQL and the ecosystem

In one line

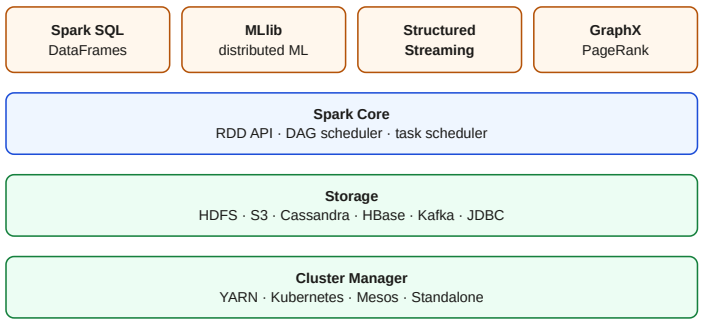
A DataFrame is a distributed table with named columns and a schema, optimised by Catalyst.

Traps

- HBase is not a Spark ecosystem component.
- DataFrames do not replace the engine: they compile down to RDDs.
- Catalyst optimises DataFrames/SQL; RDD code goes through the DAG scheduler only.

Spot it in a scenario

If the question describes...	Answer
analysts run SELECT ... GROUP BY over Parquet health records	**Spark SQL on a DataFrame**
a fraud model is trained with logistic regression across the cluster	**MLlib**
live payment events, with late data handled by watermarks	**Structured Streaming**
ranking pages by their links	**GraphX (PageRank)**
a NoSQL table that Spark reads from	**HBase: a data source, not part of Spark**



Spark SQL, MLlib, Structured Streaming and GraphX all run on Spark Core, which reads from storage and gets resources from a Cluster Manager.

Week 7

Hive, HiveQL and SQL-on-Hadoop

Hive lets analysts query data on HDFS with SQL-like HiveQL, which it compiles into MapReduce, Tez or Spark jobs, using a Metastore that records every table's schema and location. The chapter covers how Hive runs a query, schema-on-read, partitioning and bucketing for speed, Presto for interactive queries, and Spotify's Discover Weekly pipeline as a case study.

Exam questions

Multiple choice (4)

Q1 In Hive, where is table schema metadata (column names, types, partition info) stored?

(a) HDFS NameNode
(b) Hive Metastore (relational DB such as MySQL)
(c) HBase
(d) ZooKeeper

> **(b) Hive Metastore (relational DB such as MySQL)**
> The NameNode holds HDFS block metadata, not table schemas.

Q2 What does "schema-on-read" mean in the context of Hive?

(a) Schema is validated when data is inserted into the table
(b) Schema is applied when data is queried, not when it is stored on HDFS
(c) Schema is stored separately from data in HDFS metadata
(d) Schema is inferred automatically from column names in the file header

> **(b) Schema is applied when data is queried, not when it is stored on HDFS**
> (a) describes schema-on-write, how MySQL works.

Q3 Which Hive optimization technique physically organises data into separate HDFS sub-directories by a column's value to avoid full table scans?

(a) Bucketing
(b) Indexing
(c) Partitioning
(d) Compression

> **(c) Partitioning**
> Bucketing splits files inside a partition; it does not skip directories.

Q4 Presto's primary performance advantage over Hive+MapReduce for interactive queries is:

(a) Presto supports more SQL functions than HiveQL
(b) Presto has better native integration with HBase and Cassandra
(c) Presto executes queries entirely in memory without writing intermediate results to disk
(d) Presto has native Python UDF support

> **(c) Presto executes queries entirely in memory without writing intermediate results to disk**
> Hive+MR writes to HDFS between every stage.

True or false (4)

1 HiveQL supports full ACID transactions on all table types, just like MySQL.

Old slides (v1)

> **False.** Corrected: HiveQL supports ACID only on ACID-enabled transactional ORC tables; MySQL (InnoDB engine) gives full ACID: INSERT, UPDATE, DELETE and ROLLBACK on any row.
>
> **From the lecturer's files**
>
> **Model justification (old slides):** ACID exists only on ORC-backed transactional tables created with TBLPROPERTIES ('transactional'='true').

2 Partitioning in Hive reduces query time by eliminating irrelevant data partitions from the HDFS scan.

True. A WHERE filter on the partition key makes Hive read only the matching sub-directories (partition pruning).

3 Hive directly executes a HiveQL query on HDFS without any intermediate translation step.

Old slides (v1)

False. Corrected: Hive compiles HiveQL (parse, analyse, plan, optimise) into MapReduce, Tez or Spark jobs, which then read HDFS.

> **From the lecturer's files**
>
> **Model justification (old slides):** The Driver parses HiveQL, builds and optimises a logical plan, and compiles it into a DAG of MapReduce, Tez or Spark jobs before any data is touched.

4 Bucketing in Hive is particularly useful for optimising JOIN operations between two large tables.

Old slides (v1)

True. Both tables bucketed on the join key with the same N let Hive join bucket i with bucket i, with no full shuffle.

> **From the lecturer's files**
>
> **Model justification (old slides):** A bucket map join pairs matching bucket numbers (same hash), so the full shuffle-sort phase is skipped.

Descriptive (3)

D1 Explain the Hive architecture: what is the role of HiveServer2, the Driver, and the Metastore? How is a HiveQL query compiled into executable jobs? Draw the component diagram.

- **HiveServer2**: JDBC/ODBC front door; authenticates users, accepts HiveQL from Beeline, BI tools, JDBC.
- **Driver**: Parser → Semantic Analyser → Logical Planner → Optimizer → Physical Plan Generator; manages the query lifecycle.
- **Metastore**: MySQL/PostgreSQL catalogue of schemas, types, partitions, HDFS paths.

Compilation: parse to AST → check names and types in the Metastore → logical plan (Filter, Join, GroupBy, OrderBy) → optimise (partition pruning) → MR/Tez/Spark jobs → submit to YARN; results to HDFS and client.

Diagram:

See the diagram on the topic "Hive architecture and query compilation"

D2 Differentiate between partitioning and bucketing in Hive. When would you choose each? Describe a scenario in Bangladesh e-commerce data where both are used together.

	Partitioning	Bucketing
Unit	Directory per value	Fixed N files, hash(col) % N
Benefit	Pruning for WHERE filters	Bucket map join
Column	Low cardinality (date, division)	High cardinality key (customer_id)

Choose partitioning when queries filter on a few-valued column; bucketing when large tables are joined on a many-valued key.

Scenario: a Bangladeshi e-commerce `orders` table, `PARTITIONED BY (order_date, division)` and `CLUSTERED BY (customer_id) INTO 32 BUCKETS`. The Dhaka daily report reads one folder; the join with `customers` (also 32 buckets on customer_id) pairs bucket i with i.

D3 What is "schema-on-read"? What are its advantages and disadvantages compared to "schema-on-write" in a traditional RDBMS such as MySQL? Give one example where schema-on-read causes a data quality problem.

Schema-on-read: Hive applies column types at query time; loading is a raw file copy. MySQL (schema-on-write) checks each row at INSERT.

Advantages	Disadvantages
Fast load, no validation	Slower queries (parse every time)
Load before the schema is known; many schemas on the same files; no ETL at ingest	Bad data silent: becomes NULL
Open formats (ORC, Parquet, CSV, JSON, Avro)	Unfit for OLTP (banking, orders)

Example: `abc` in a DOUBLE `amount` column. MySQL rejects the INSERT; Hive stores it and returns NULL, so `SUM(amount)` under-reports revenue with no warning. Fix: Great Expectations or dbt tests downstream.

Analytical (2)

A1 Partitioning & Bucketing Design: You have a 5 TB e-commerce transaction table with columns: (txn_id, user_id, product_id, amount, country, txn_date). Design a complete partitioning and bucketing strategy to optimise: – Daily revenue reports filtered by country and txn_date – JOINs between this table and a 10 M-row user profile table (keyed on user_id). Write the complete CREATE TABLE DDL. Justify why you chose a specific number of buckets and which column(s) you partitioned on.

★ Very likely | Lecturer's plan | CLO4

Partition on `txn_date, country` (low cardinality, used in report filters). **Bucket** on `user_id` (high-cardinality join key), same N on both tables for a bucket map join. Never partition on user_id or txn_id.

Bucket count (assume 3 years, 5 countries):

Partitions	3 × 365 × 5 = 5,475
Per partition	5,000 GB / 5,475 ≈ 0.913 GB
8 buckets	≈ 114 MB/file (near 128 MB block), 43,800 files
32 buckets	≈ 28.5 MB/file, 175,200 files: too small

```
CREATE TABLE transactions (
  txn_id BIGINT, user_id BIGINT,
  product_id BIGINT, amount DOUBLE)
PARTITIONED BY (txn_date STRING, country STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;

CREATE TABLE user_profile (
  user_id BIGINT, name STRING, city STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;
```

Result: a one-day, one-country report reads 1 of 5,475 partitions (≈ 0.018%); the join pairs bucket i with i.

A2 Query Compilation to MapReduce: Write HiveQL to find the top 5 products by total revenue in 2023. Then trace how Hive compiles this query into MapReduce stages: what is emitted by the Map phase, what is the shuffle key, and what does the Reduce phase compute? How would switching the execution engine from MapReduce to Tez change the execution plan?

★ Very likely | Lecturer's plan | CLO4

```
SELECT product_id, SUM(amount) AS total_revenue
FROM transactions
WHERE YEAR(txn_date) = 2023
GROUP BY product_id
```

```
ORDER BY total_revenue DESC
LIMIT 5;
```

	Job 1: aggregate	Job 2: sort + limit
Map emits	(product_id, amount) for 2023 rows; combiner gives partial sums	(total_revenue, product_id); each mapper keeps its top 5
Shuffle key	product_id	total_revenue DESC, to one reducer
Reduce	SUM(amount) per product, written to HDFS	First 5 rows

Tiny check: (P1,100), (P2,50), (P1,30), (P3,80), (P2,20) → P1 130, P3 80, P2 70.

Tez: one DAG, Map 1 → Reducer 2 → Reducer 3. No HDFS write between the jobs, one submission, container reuse. Same result, faster.

> **From the lecturer's files**
>
> The plan also asks for the monthly average per country in Q1 2024; see that worked query in Practise.

Discussion (3)

1 Spotify's co-occurrence query runs GROUP BY song_a, song_b over 2 billion playlist rows. Estimate the output size if there are 30M songs. Propose a partitioning + bucketing scheme for the playlist_songs Hive table that would reduce the shuffle size for this query. [CLO4 – Apply]

★ Very likely | Old slides (v1) | CLO4

Claim: output is far below the 30M-song bound. Bound: 30M × (30M – 1) / 2 ≈ 4.5 × 10^14 pairs. Realistic (assume 20 songs per playlist): 10^8 playlists × 190 = 1.9 × 10^10 pairs, ≤ 304 GB at 16 B/row, much less after HAVING > 5. (If you read it as 2 billion playlists of 20 songs: 2 × 10^9 × 190 = 3.8 × 10^11 pairs, ≈ 6.1 TB at 16 B; state which reading you use.)

Scheme: `PARTITIONED BY (week)`, `CLUSTERED BY (playlist_id) SORTED BY (playlist_id, song_id) INTO 1024 BUCKETS`. **For:** the self-join runs inside each bucket with no shuffle, and weekly partitions cut 8 h to 2 h. **Against:** it does not shrink the GROUP BY shuffle on song pairs; rely on map-side aggregation there.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

2 The ALS collaborative filtering model is a matrix factorisation over a 75M-user × 30M-song matrix. At 4 bytes per float and rank k = 50, calculate the model size and explain whether it fits in a 5,000-node YARN cluster's aggregate RAM. [CLO4 – Analyse]

★ Very likely | Old slides (v1) | CLO4

Size: (75M + 30M) × 50 × 4 B = **21 GB** (users 15 GB + songs 6 GB).

Fits? Assume 64 GB/node: 5,000 × 64 GB = 320 TB; 21 GB ≈ 0.0066%, about 4.2 MB per node. **For:** it fits easily, even on one large server; the dense 9 PB matrix would not. **Against:** training also holds the interaction data, shuffled factor blocks and JVM overhead, several times the model size, but still well within the cluster.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

3 Spotify uses Luigi while other companies use Apache Airflow. Compare the two on: dependency declaration style, backfill capability, and UI/observability. Which would you choose for a 100-step daily pipeline? [CLO3 – Evaluate]

Old slides (v1) CLO3

Claim: choose Airflow for a 100-step daily pipeline. Luigi: target-based `requires()/output()`, skips finished outputs, backfill by date parameters, simple UI. Airflow: DAG with a scheduler, built-in catchup/backfill, rich UI with logs, retries and alerts.

For: at 100 steps you need to see which step failed, read its logs and re-run past days, which Airflow gives out of the box. **Against:** Luigi is lighter and its skip-if-output-exists model fits file-based Hadoop pipelines like Spotify's.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

Live exercises (5)

Task Task (25 minutes, pairs): Simulate a Hive session using PySpark's SQL interface (which reads the same Hive Metastore). Dataset: a 10,000-row synthetic play_events CSV (generate with the skeleton below).

```
from pyspark.sql import SparkSession
import random, datetime

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# Generate synthetic data
rows = []
for i in range(10_000):
    rows.append({
      "user_id":    random.randint(1, 500),
      "song_id":    random.randint(1, 200),
      "playlist_id":random.randint(1, 1000),
      "dt":         "2024-10-01"
    })

df = spark.createDataFrame(rows)
df.createOrReplaceTempView("play_events")

# Co-occurrence query
result = spark.sql("""
    SELECT a.song_id AS song_a,
           b.song_id AS song_b,
           COUNT(*) AS co_count
    FROM play_events a
    JOIN play_events b
      ON a.playlist_id = b.playlist_id
     AND a.song_id < b.song_id
    GROUP BY a.song_id, b.song_id
    HAVING COUNT(*) > 2
    ORDER BY co_count DESC
""")
result.show(10)
```

Write the data as a real partitioned table, then time and inspect the queries.

```
import time
random.seed(42)  # rows as in the skeleton, dt from 2 dates
df.write.mode("overwrite").partitionBy("dt") \
  .format("parquet").saveAsTable("play_events_part")
t0 = time.time()
spark.sql(cooc_sql).show(10)      # skeleton query on play_events_part, with a.dt = b.dt = '2024-10-01'
print(time.time() - t0)
songs = spark.createDataFrame(
```

```
    [(i, "genre" + str(i % 10)) for i in range(1, 201)], ["song_id", "genre"])
songs.createOrReplaceTempView("songs")   # 200-row dimension table
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
hint_sql = """SELECT /*+ BROADCAST(s) */ s.genre, COUNT(*)
  FROM play_events_part p JOIN songs s ON p.song_id = s.song_id
  WHERE p.dt = '2024-10-01' GROUP BY s.genre"""
spark.sql(hint_sql).explain(mode="formatted")
```

Expected with the single-date skeleton: about 10 rows per playlist, about 50,000 pair rows, about 9,000 pairs kept by HAVING > 2, top count about 11 to 14 (simulated; varies with the data).

1 Create a partitioned Spark table (partition on dt).

`df.write.partitionBy("dt").saveAsTable("play_events_part")` writes one sub-directory per date, e.g. `play_events_part/dt=2024-10-01/`, like Hive's `year=2024/`. Expected `SHOW PARTITIONS`: one row per date.

2 Run the co-occurrence query and time it.

Wrap an action (`show`, `count`) in `time.time()`: Spark is lazy, so `spark.sql()` alone runs nothing. Expected: a few seconds on a laptop, mostly start-up and the join/GROUP BY shuffle. The self-join grows with the square of playlist length (10 songs give 45 pairs).

3 Add a broadcast join hint for the songs dimension table and compare timings.

`/*+ BROADCAST(s) */` copies the 200-row songs table to every executor, so the big table is not shuffled (Hive's map-side join). The plan changes from SortMergeJoin to BroadcastHashJoin. Expected: slightly faster on 10,000 rows. Spark broadcasts tables under 10 MB automatically, so set the threshold to -1 to compare.

4 Use EXPLAIN to print the physical plan and identify where partition pruning occurs.

Run `EXPLAIN FORMATTED` or `df.explain(mode="formatted")`. Pruning shows in the FileScan node as `PartitionFilters: [isnotnull(dt), (dt = 2024-10-01)]`: only that folder is read. Filters on ordinary columns appear as `PushedFilters`. A temp view built in memory shows no PartitionFilters.

Topics

Hive architecture and query compilation

In one line

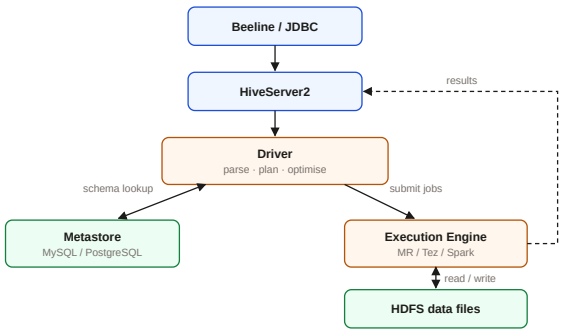
Hive is a data warehouse layer on Hadoop that compiles SQL-like HiveQL queries into MapReduce, Tez or Spark jobs over files on HDFS.

Traps

- Hive never runs HiveQL directly on HDFS. It compiles the query to MR, Tez or Spark jobs first.
- Schemas live in the Metastore, not the NameNode. The NameNode only knows files and blocks.
- Switching to Tez changes the plan and the speed, never the answer.
- ORDER BY needs one reducer for a global order, so the final sort stage is a bottleneck.

Spot it in a scenario

If the question describes...	Answer
Tableau connects over JDBC to run HiveQL	**HiveServer2**
a query fails with 'column not found' before any job starts	**Semantic analysis against the Metastore (Driver)**
every HDFS file is intact but no table can be queried	**Metastore failure**
one SET command makes the same query faster with no HDFS writes between stages	**Engine switched from MapReduce to Tez or Spark**
each mapper sums counts per country before the shuffle	**Map-side aggregation (combiner)**



A HiveQL query enters through HiveServer2; the Driver looks up the schema in the Metastore and submits jobs to the execution engine, which reads and writes HDFS.

Worked example

P2 slide 10: `SELECT country, COUNT(*) FROM logs GROUP BY country`, 50 million rows, 200 countries. Without map-side aggregation the mappers emit one (country, 1) row per input row: 50,000,000. With it they emit one partial count per country: 200. Reduction = 50,000,000 / 200 = **250,000×** less shuffle data.

Step by step

1. Given 50,000,000 rows and no combiner → shuffle rows = 50,000,000
2. Given 200 countries and one mapper with map-side aggregation → shuffle rows = 200
3. Given 50,000,000 and 200 → reduction = 50,000,000 / 200 = 250,000×
4. Given M mappers that each see all 200 countries → shuffle = M × 200 rows, reduction = 50,000,000 / (M × 200)

Variants

- 120 million rows, 400 regions: 120,000,000 / 400 = 300,000×
- 50 million rows, 10 mappers each seeing all 200 countries: shuffle = 2,000 rows, reduction = 25,000×

Schema-on-read and HiveQL vs SQL

In one line

Schema-on-read means Hive applies the table's column types when data is queried, not when it is loaded.

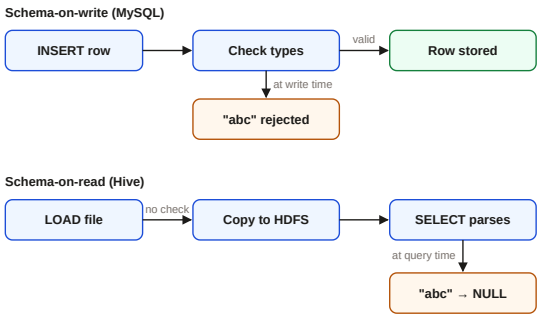
Traps

- Schema-on-read does not mean the schema is inferred. You still write CREATE TABLE (MCQ 2 option d).
- The schema is stored separately under both models. The key word is *when* it is applied.
- HiveQL looks like SQL but runs as batch jobs: minutes, not milliseconds. A `COUNT(*)` on 1 TB can take 5–10 min on MapReduce; use Presto or Spark SQL for interactive speed.

  Exam Tip (old slides v1); still on current slide 17 ★ Very likely
- Hive ACID is not on all tables, only transactional ORC tables.

Spot it in a scenario

If the question describes...	Answer
raw clickstream logs are loaded today; the schema will be decided next month	**Schema-on-read**
a revenue report is too low with no error; some amount values were text	**Schema-on-read silent NULLs**
malformed money transfers must be refused the moment they arrive	**Schema-on-write (RDBMS, OLTP)**
an analyst expects a millisecond reply from Hive COUNT(*) on 1 TB	**Misconception: Hive is batch; use Presto or Spark SQL**
someone runs UPDATE on a plain CSV Hive table	**Not allowed: needs an ACID transactional ORC table**



Schema-on-write checks types at INSERT and rejects bad rows; schema-on-read copies the file unchecked and turns a bad value into NULL at query time.

Partitioning and partition pruning

In one line

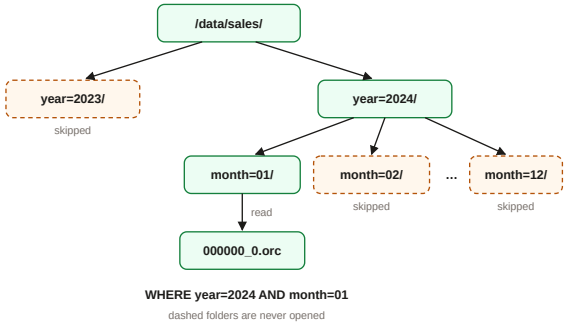
Partitioning stores a Hive table in separate HDFS sub-directories by the value of a column, so queries filtering on it read only matching directories.

Traps

- Partitioning splits by directory; bucketing splits into files inside a partition. Only partitioning skips directories.
- In a dynamic insert, partition columns must be last in SELECT and in PARTITION order, or Hive assigns values wrongly without error.
- Pruning needs a filter on the partition column itself. A filter on any other column scans every partition.
- The fraction formula assumes equal-size partitions. Say so.

Spot it in a scenario

If the question describes...	Answer
daily reports filter on date but scan the whole table	**Partition by date**
a designer proposes partitioning a 50M-customer table by customer_id	**Partition explosion; bucket on customer_id instead**
36 months are loaded into their partitions in one INSERT	**Dynamic partition insert**
after a dynamic insert, month values sit in the year= folders	**Partition columns out of order in SELECT**
EXPLAIN shows `PartitionFilters: [dt = 2024-10-01]` in the scan	**Partition pruning**



With WHERE year=2024 AND month=01, Hive reads only the month=01/ folder and never opens the other partition directories.

Worked example

P2 slide 11: `play_events` partitioned by dt (365 days) and country (200). P_total = 365 × 200 = 73,000. The query reads dt='2024-10-01' for BD, IN, US: P_read = 3. Fraction read = 3 / 73,000 ≈ **0.0041%**; 72,997 partitions skipped. P1 slide 19: one month of 36 = 1/36 ≈ 2.78%.

Step by step

1. Given 365 dates and 200 countries → P_total = 365 × 200 = 73,000
2. Given dt = one day and country IN (BD, IN, US) → P_read = 1 × 3 = 3
3. Given 3 of 73,000 (equal-size partitions) → fraction = 3 / 73,000 ≈ 0.0041%
4. Given 73,000 total and 3 read → skipped = 72,997

Variants

- 36 months, query one month: 1 / 36 ≈ 2.78% read
- 24 months × 50 countries, one month for 2 countries: 2 / 1,200 ≈ 0.167% read

Bucketing and bucket map join

In one line

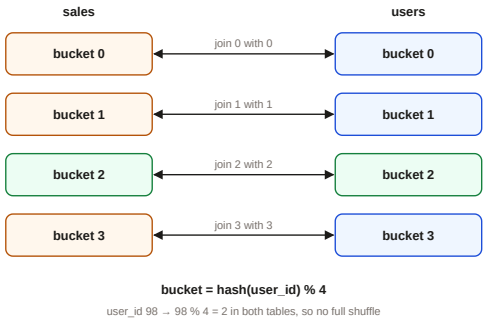
Bucketing splits each Hive partition into a fixed number of files by hash(column) % N.

Traps

- Bucketing does not skip directories. It cuts join cost.
- Buckets are numbered from 0: with 4 buckets the files are 0 to 3.
- Same key and same bucket count on both tables, or no bucket map join.
- The 5 TB question gives no years or countries. State your assumptions before sizing.

Spot it in a scenario

If the question describes...	Answer
two huge tables are joined every day on customer_id	**Bucket both on customer_id with the same N**
a join still shuffles although both tables are bucketed, one on user_id and one on email	**Different bucket keys: no bucket map join**
find which file holds user_id 98 with 4 buckets	**98 % 4 = bucket 2**
a 5 TB fact table needs daily reports by date and country plus joins on user_id	**Partition by txn_date, country; bucket by user_id**



Both tables bucketed on user_id into 4 buckets, so bucket i joins only bucket i (bucket map join) with no full shuffle.

Worked example

Bucket number (4 buckets on user_id, as in the P1 slide 24 diagram; user_id 98 is our own example): 98 % 4 = **2**, in both sales and users.

File size (A1, assume 3 years and 5 countries): P = 3 × 365 × 5 = 5,475 partitions; 5 TB / 5,475 ≈ 0.913 GB each; with 8 buckets ≈ **114 MB** per file (near a 128 MB block), 5,475 × 8 = 43,800 files.

Step by step

1. Given user_id = 98, N = 4 → bucket = 98 % 4 = 2
2. Given 3 years, 365 days, 5 countries → P = 1,095 × 5 = 5,475
3. Given 5 TB = 5,000 GB and P = 5,475 → ≈ 0.913 GB per partition
4. Given 0.913 GB and N = 8 → ≈ 114 MB per bucket file
5. Given P = 5,475 and N = 8 → files = 43,800

Variants

- user_id 1,234,567 with 32 buckets: 1,234,567 % 32 = 7
- N = 32 in the 5 TB design: ≈ 28.5 MB per file and 175,200 files: too small
- 2 TB, 730 daily partitions, 16 buckets: 2,000 GB / 11,680 ≈ 171 MB per file

Presto and the SQL-on-Hadoop engines

In one line

Presto is Meta's in-memory SQL engine that answers interactive queries over HDFS in seconds by never writing intermediate results to disk.

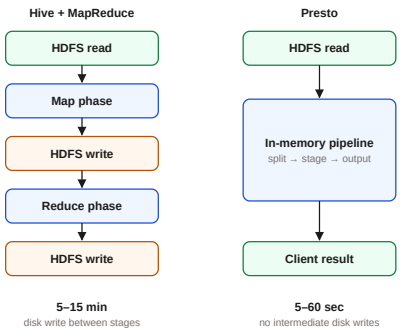
Traps

- Presto's speed comes from no intermediate disk writes, not from extra SQL functions, connectors or Python UDFs.
- Presto replaces the engine, not the catalogue: it reads the Hive Metastore.
- The slide title says 10× faster; the ranges (5–15 min vs 5–60 s) give 5× to 180×.

Spot it in a scenario

If the question describes...	Answer
a dashboard over Hive tables must answer in seconds	**Presto/Trino (or Spark SQL)**
a nightly 6-hour ETL over 50 TB	**Hive**

If the question describes...	Answer
one query joins Kafka data with a MySQL table and S3 files	**Presto connectors**
SQL over raw JSON files with no schema defined	**Apache Drill**
an engine that processes 1,024-row column batches instead of single rows	**Vectorized execution (Presto)**



Hive+MapReduce writes to HDFS between stages and takes minutes; Presto streams data through an in-memory pipeline and answers in seconds.

Case study: Spotify Discover Weekly

In one line

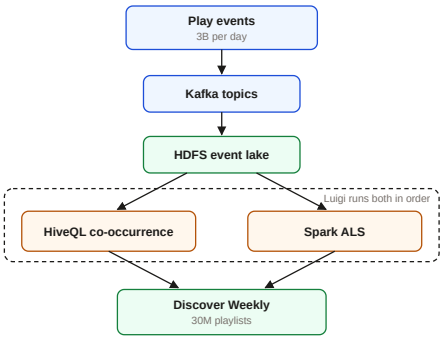
Discover Weekly is Spotify's weekly recommendation playlist, built from playlist co-occurrence in HiveQL and ALS collaborative filtering in Spark, scheduled by Luigi.

Traps

- ALS size uses users *plus* songs, not users *times* songs.
- The slides give no RAM per node. State the assumption (e.g. 64 GB).
- Bucketing playlist_songs by playlist_id speeds the self-join, not the GROUP BY on song pairs.

Spot it in a scenario

If the question describes...	Answer
a re-run skips every step whose HDFS output already exists	**Luigi (target-based)**
a song released yesterday gets no playlist-based recommendations	**Cold start**
50 taste scores per user and per song replace the full rating matrix	**ALS matrix factorisation**
the same HiveQL got 4× faster after a table redesign	**Partitioning play_events by week**



Play events flow through Kafka to HDFS, where Luigi runs the HiveQL co-occurrence and Spark ALS jobs that build Discover Weekly.

Worked example

ALS model size (Discussion 2): (75M + 30M) × 50 × 4 B = 2.1 × 10^10 B = **21 GB**. The full matrix would be 75M × 30M × 4 B = 9 × 10^15 B = 9 PB. **Speed-up** (lesson 3): 8 h / 2 h = **4×**, 75% time

saved.

Step by step

1. Given 75M users, k = 50, 4 B → user factors = 75 × 10^6 × 50 × 4 = 15 GB
2. Given 30M songs → song factors = 30 × 10^6 × 50 × 4 = 6 GB
3. Given 15 GB + 6 GB → model = 21 GB
4. Given 5,000 nodes × 64 GB (assumed) → 320 TB RAM; 21 GB ≈ 0.0066% of it: fits
5. Given 8 h before, 2 h after → speed-up = 4×, saved = (8 − 2) / 8 = 75%
6. Given a playlist of L = 20 songs → pairs = 20 × 19 / 2 = 190

Variants

- 10M users, 1M songs, k = 100: 11M × 100 × 4 B = 4.4 GB
- Dense matrix 9 PB vs 320 TB RAM: about 28× too big, does not fit
- 2B (playlist, song) rows at 20 songs each: 10^8 playlists × 190 = 1.9 × 10^10 pairs

Week 8

Data integration: Kafka, Lambda/Kappa, data lakes

How data moves from the systems that create it to the systems that analyse it: batch tools (Sqoop, Flume) and the Kafka event log. How Lambda and Kappa architectures combine batch and stream processing, and how a data lake stays usable instead of turning into a swamp.

Exam questions

Multiple choice (4)

Q1 Which tool is specifically designed to transfer data in bulk between an RDBMS (e.g., MySQL) and HDFS?

(a) Apache Flume
(b) Apache Kafka
(c) Apache Sqoop
(d) Apache NiFi

> **(c) Apache Sqoop**
> Sqoop (SQL-to-Hadoop) bulk-copies tables between a database and HDFS; Flume is for logs.

Q2 Apache Kafka's data model is based on which fundamental storage abstraction?

(a) B-tree indexes for fast point lookups
(b) Column-family storage (like HBase)
(c) Append-only immutable distributed logs (partitioned topics)
(d) Document storage with secondary indexes

> **(c) Append-only immutable distributed logs (partitioned topics)**
> Kafka is a distributed, partitioned, replicated commit log.

Q3 In the Lambda architecture, which layer handles **real-time**, **low-latency** query views?

(a) Batch Layer
(b) Serving Layer
(c) Speed Layer
(d) Ingestion Layer

> **(c) Speed Layer**
> The Serving layer only merges views; the Speed layer computes the seconds-old views.

Q4 The Kappa architecture differs from the Lambda architecture primarily by:

(a) Adding a third processing layer for machine learning
(b) Using only relational databases for both batch and stream storage
(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
(d) Replacing Hadoop with Spark in the batch layer while keeping the speed layer

> **(c) Eliminating the batch layer and using Kafka log replay for all reprocessing**
> Kappa removes the whole batch layer; (d) is the common misconception.

True or false (4)

1 Apache Sqoop is designed for real-time streaming ingestion from transactional systems.

Old slides (v1)

> **False.** Sqoop runs batch MapReduce imports. Corrected: Sqoop is designed for bulk batch transfer between an RDBMS and HDFS; real-time ingestion uses Kafka.
>
> **From the lecturer's files**
>
> **Model justification (old slides):** Sqoop is a batch tool that launches MapReduce jobs for bulk RDBMS ↔ HDFS transfers, usually nightly; real-time ingestion is Kafka's job.

2 Kafka retains messages for a configurable period, allowing consumers to replay events from any past offset.

> **True.** Messages stay for the retention period (e.g. 7 days) after being read, so a consumer can reset its offset and replay.

3 The Lambda architecture requires maintaining two separate code paths implementing the same business logic.

Old slides (v1)

True. The same logic is written for the batch layer (Spark/Hive) and the speed layer (Spark Streaming) and must be kept in sync.

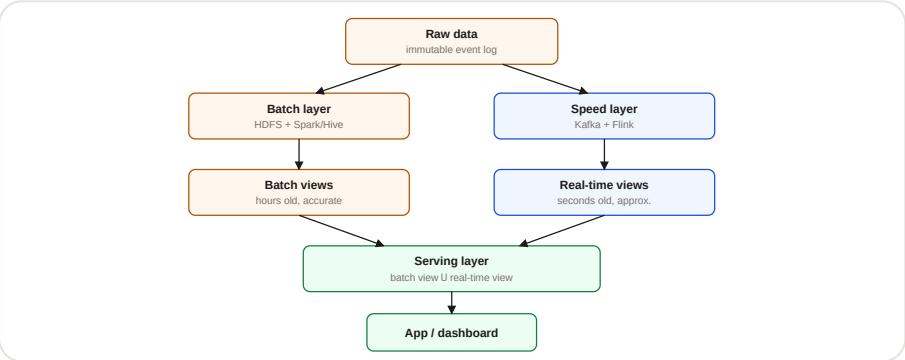
> **From the lecturer's files**
>
> **Model justification (old slides):** Both layers implement the same aggregation (e.g. daily transaction count), once in batch (Spark/HiveQL) and once in streaming (Spark Streaming/Flink); this dual maintenance is the main engineering cost.

4 Apache Flume is the preferred tool for collecting and aggregating server log data into HDFS.

True. Flume moves logs into HDFS through Source, Channel and Sink with fan-in from many servers.

Descriptive (3)

D1 Explain the **Lambda architecture** with a labelled diagram. What are its three layers? What is its primary engineering disadvantage compared to the Kappa architecture?



- **Batch:** master dataset on HDFS, recomputed hourly/nightly; accurate, 1–24 h stale.
- **Speed:** stream processing of data since the last batch; approximate, results within seconds.
- **Serving:** query = batch view UNION real-time view.

Disadvantage vs Kappa: two code paths for the same logic, kept in sync (30–40% of effort), plus two clusters and a runtime merge. Kappa has one stream code path.

D2 Describe how Apache Kafka achieves **high throughput and fault tolerance**. Explain the roles of topics, partitions, offsets, brokers, and replication factor in the Kafka data model.

Term	Role
Topic	Named append-only log for one event type
Partition	Ordered slice of a topic; P partitions run in parallel; same key → same partition
Offset	Message position; each consumer tracks its own
Broker	Server storing partitions
Replication factor R	1 leader + R – 1 followers per partition (default 3)

Throughput: sequential writes, zero-copy sendfile(), batching, partition parallelism.

Fault tolerance: a follower takes over when a leader's broker fails; retention lets a crashed consumer resume from its offset. Example: bkash-txns, 3 partitions, RF = 2 survives 1 broker failure.

D3 What is a **data lake**? How does it differ from a data warehouse in terms of schema enforcement, storage format, and typical use cases? What is a "data swamp" and how is it prevented?

Data lake: central store of raw data in any format, no upfront schema.

	Lake	Warehouse
Schema	On read	On write
Format	Raw files, any type	Cleaned relational tables
Use	ML, data science, exploration	BI reports, dashboards

Data swamp: a lake with no governance: no catalogue, stale vs current unknown, unversioned copies, PII mixed in.

Prevention: catalogue at ingest, owner per dataset, access control, quality checks, Delta Lake/Iceberg for ACID.

Analytical (2)

A1 **Pipeline Design for a Bank:** Design a data integration pipeline for a Bangladeshi commercial bank that must:

- (a) Synchronise 10 million customer records nightly from Oracle to Hadoop for analytics
- (b) Detect fraudulent card-swipe events in real time with a decision within 200 ms
- (c) Power a risk management dashboard with data no older than 5 minutes

Specify which tool (Sqoop, Flume, Kafka, Spark Streaming) you would use at each stage. Justify each choice with reference to latency, throughput, and fault-tolerance requirements. Which architecture (Lambda or Kappa) fits best, and why?

★ Very likely | Lecturer's plan | CLO4

Need	Tool	Latency	Throughput	Fault tolerance
(a) 10 M rows nightly	Sqoop (incremental import)	Hours is fine: batch	Parallel map tasks by key range, GB–TB/h	Failed map task retried; HDFS replicated
(b) Fraud < 200 ms	Kafka topic (keyed by card no.) + Spark Streaming	Milliseconds	Partitions scale consumers	RF = 3; restart from last offset
(c) Dashboard < 5 min	Second consumer group on same topic + Spark Streaming aggregates	Minutes	Same topic, no extra ingest	Replay from offset

Flume is not used: it is for logs, not tables or card events.

Architecture: Lambda. The bank has a real batch need (nightly sync, regulatory batch reports), which the slide lists as a reason Kappa is not suitable. Batch = Sqoop + HDFS + Hive; Speed = Kafka + Spark Streaming; Serving = dashboard. Cut the dual-path cost by making Kafka the single source both layers read.

A2 **Lambda to Kappa Migration:** A team using Lambda architecture complains that maintaining two code paths (batch + stream) doubles development cost. Their CTO suggests migrating to Kappa. Analyse: (a) what must be true about the stream processing engine for Kappa to be viable; (b) what are the risks of migrating; (c) under what conditions is Kappa **not suitable** (give at least two concrete scenarios from the Bangladesh financial or healthcare sector)?

★ Very likely | Lecturer's plan | Old slides (v1) | Lecturer's textbook ch 8 | CLO4

(a) Conditions: the engine reprocesses history about as fast as live data; full history sits in a replayable log (long Kafka retention or Iceberg on S3; default is only 7 days); the logic fits a stream job; exactly-once processing so replays match.

(b) Risks: TB+ replay may take days; batch throughput falls to stream-engine speed; storage cost of long retention; every Hive/Spark batch job must be rewritten and run in parallel during the switch; staff must learn stream SQL.

(c) Not suitable:

1. **Regulatory reconciliation:** Bangladesh Bank requires a daily end-of-day batch reconciliation; the batch layer must stay.
2. **TB+ history:** a national health-records system recomputing 10 years of disease statistics after a coding change; one Spark batch job is far faster than replay.
3. **Batch SQL preferred:** yearly credit-risk model training written in Hive/Spark SQL over full history.

Verdict: move streaming workloads to one Kappa path; keep batch where regulation or huge history demands it.

> **From the lecturer's files**
>
> The old (v1) slides asked the same thing: name two workloads where Lambda beats Kappa. Textbook (ch 8) numbers: replaying 5 PB of Kafka history at 1 GB/s = 5,000,000 GB ÷ 1 GB/s = 5,000,000 s ≈ 58 days, while a Spark batch job on 200 nodes finishes in hours. Payment rules (e.g. EU PSD2) also demand a separate batch reconciliation.

Discussion (5)

L1 **Kafka as the single source of truth**: all events written to Kafka first; the speed and batch layers are both consumers of the same topic — eliminating the data inconsistency problem of classic Lambda (two separate ingestion paths)

> **Claim:** Netflix writes each event to Kafka once; Flink (speed, < 1 min) and the S3 sink + Spark/Iceberg (batch, ~1 day) read the same topic, so both layers see the same data. **For:** no mismatch between layers; new consumers just subscribe. **Against:** Kafka becomes a single critical dependency (2,000+ brokers to keep up), and the two code paths still exist; only ingestion is unified.

L2 **Retention enables reprocessing**: 7-day retention lets Netflix replay any event window when a Flink job has a bug or a Spark job fails

> **Claim:** reading does not delete events, so a fixed job resets its offset and recomputes the last week. **For:** bugs are fixed after the fact without asking devices to resend. **Against:** only 7 days; a bug found on day 8 needs the S3/Iceberg copy. At 100 billion events/day, 7 days is 700 billion events of storage; tiered storage (Kafka 3.6) moves old segments to S3.

L3 **Kappa simplification**: Netflix increasingly moved batch jobs to Flink stream processing (processing historical S3 data as a bounded stream), reducing pipeline complexity

> **Claim:** a bounded stream (data with a start and an end, such as last month's S3 files) can be read by the same Flink code as live data, so one code path covers both. **For:** less duplicate code, fewer systems. **Against:** Netflix still keeps Spark + Iceberg for ML training and billing, where batch is easier or cheaper; the move is gradual.

L4 **Schema Registry is non-negotiable at scale**: 4,000 topics × evolving schemas ⇒ consumer breakage without schema governance

> **Claim:** a schema is a message's field names and types; if a producer renames a field, consumers break. Schema Registry stores the Avro/Protobuf schema per topic so consumers can read any past producer version. **For:** 4,000+ topics and 150+ Flink jobs cannot be coordinated by hand. **Against:** another service to run, and teams must follow compatibility rules. It is the streaming form of the data-swamp lesson.

L5 **Backpressure is the real production problem**: during Super Bowl commercials, event rates spike 5×; Kafka's durable log absorbs the burst; Flink consumers catch up within minutes

> **Claim:** backpressure is data arriving faster than consumers can process. If the 5× spike landed on the 1.3 M/s peak, that would be 6.5 M/s; Kafka appends the excess to disk and Flink catches up later. **For:** no data lost, no need to size consumers for the rare peak. **Against:** results lag by minutes during the spike (the 5-minute A/B latency slips), and brokers need spare disk.

Live exercises (5)

Task Simulate a Kafka producer/consumer pipeline using the **kafka-python** library with a local single-broker cluster (Docker Compose provided below).
Quick-start: `pip install kafka-python` ; `docker run -p 9092:9092 apache/kafka:3.7.0`

```
import json, random, time
from kafka import KafkaProducer, KafkaConsumer

TOPIC   = "play-events"
BROKER  = "localhost:9092"
CONTENT = [f"show_{i}" for i in range(20)]

# --- Producer ---
producer = KafkaProducer(
  bootstrap_servers=BROKER,
  value_serializer=lambda v:
      json.dumps(v).encode())

for _ in range(1_000):
    event = {
      "user_id":    random.randint(1, 500),
      "content_id": random.choice(CONTENT),
      "action":     random.choice(
                      ["play","pause","seek"]),
      "ts":         int(time.time())
    }
    producer.send(TOPIC, event)
producer.flush()
print("Produced 1,000 events")

# --- Consumer ---
consumer = KafkaConsumer(
  TOPIC,
  bootstrap_servers=BROKER,
  group_id="cse4345-group",
  auto_offset_reset="earliest",
  value_deserializer=lambda m:
      json.loads(m.decode()))

counts = {}
for msg in consumer:
    cid = msg.value["content_id"]
    counts[cid] = counts.get(cid, 0) + 1
    if sum(counts.values()) >= 1_000:
        break
print(sorted(counts.items(),
             key=lambda x: -x[1])[:5])
```

> The producer sends 1,000 JSON events (user 1–500, one of 20 shows, play/pause/seek); `flush()` waits until queued messages are sent. The consumer (group `cse4345-group`, `auto_offset_reset="earliest"`) reads from the start, counts events per `content_id`, stops after 1,000 and prints the top 5 shows. Broker: `docker run -p 9092:9092 apache/kafka:3.7.0`.
>
> Replay (goal 3): `consumer.seek_to_beginning(*consumer.assignment())`. New group (goal 4): change `group_id`.

1 Run the producer — send 1,000 synthetic Netflix play events to topic `play-events`.

> Expected output: `Produced 1,000 events`. With one partition the messages sit at offsets 0–999. Running it again appends offsets 1,000–1,999; nothing is overwritten.

2 Run the consumer — read all messages, count plays per `content_id`.

> Expected: 1,000 events read. The slide's consumer counts every event per `content_id` (play, pause and seek alike), so each of the 20 shows gets about 1,000 / 20 = **50**; the top-5 print shows the largest. To count plays only, add `if msg.value["action"] == "play"` : about 1,000 / 3 ≈ 333 plays, about 16.7 per show. Exact counts change each run because the producer is random.

3 Reset the consumer offset to the *beginning* and re-consume — verify idempotent results.

> After `seek_to_beginning` the same group re-reads all 1,000 events and gets identical counts. Reading did not delete anything, so the result is idempotent (same answer when repeated). If the producer ran again in between, there are 2,000 events: that is new data, not a broken replay.

4 Change the consumer group ID and observe that a new group starts from offset 0.

> A new group has no committed offset, so with `auto_offset_reset="earliest"` it reads all 1,000 events from offset 0. The old group's offset is untouched. Trap: kafka-python's default is "latest", which makes a new group read 0 events.

Topics

From point-to-point ETL to event pipelines

In one line

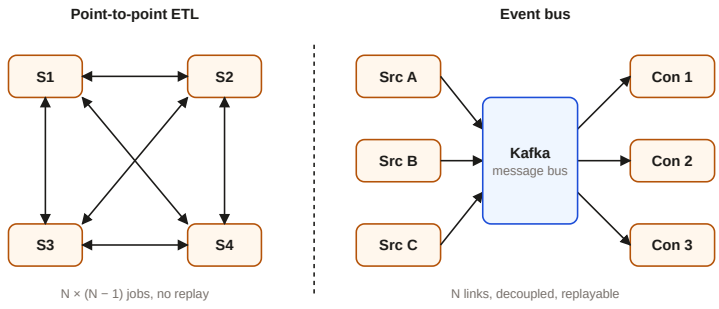
Point-to-point ETL connects every pair of systems with its own job, giving $O(N^2)$ jobs.

Traps

- The slide's 2,450 counts A→B and B→A as separate jobs. If you count each pair once, say so and divide by 2.
- Spark Streaming processes streams; it does not ingest from a database. Do not list it as an ingestion tool.
- NiFi is the visual tool, not Flume.

Spot it in a scenario

If the question describes...	Answer
every new report needs a new nightly job from each source system	**point-to-point ETL; replace with a message bus**
a consumer was down overnight and its data is gone	**no replay: a fault of point-to-point ETL**
a new analytics team plugs in without touching the source apps	**event-driven pipeline (decoupling)**
an operator wants to route IoT files by dragging boxes, no code	**Apache NiFi**



Point-to-point ETL wires every pair of systems (N × (N − 1) jobs); an event bus needs one link per system and lets consumers replay.

Worked example

Slide: 50 systems. Point-to-point jobs = 50 × 49 = **2,450**. With a bus: **50** connections.

Step by step

1. Given N = 50 systems -> each system sends to N − 1 = 49 others
2. Given 50 senders × 49 targets -> 2,450 jobs (ordered pairs)
3. Given 2,450 ordered jobs -> one job per pair = 2,450 / 2 = 1,225

4. Given a bus -> connections = N = 50

Variants

- N = 10: 10 × 9 = 90 jobs (45 if each pair is counted once), bus 10
- N = 20: 20 × 19 = 380 jobs, bus 20
- N = 30: 30 × 29 = 870 jobs, bus 30

Batch ingestion: Sqoop and Flume

In one line

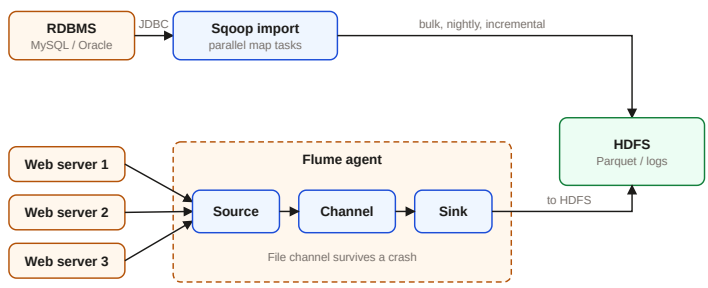
Sqoop is a batch tool that copies tables in bulk between an RDBMS and HDFS.

Traps

- Sqoop is batch, not real-time streaming (pool T/F 1 is False).
- Memory channel is faster but loses buffered events on a crash; File channel survives.
- Flume is for logs, not database tables. Database → HDFS is Sqoop.

Spot it in a scenario

If the question describes...	Answer
nightly at 01:00, copy 10 M wallet transactions from Oracle into HDFS as Parquet	**Sqoop import (bKash)**
only copy rows added since yesterday's run	**Sqoop incremental import (check-column + last-value)**
push Spark results back into MySQL	**Sqoop export**
three web servers' access.log files collected into /logs/YYYY/MM/DD/	**Flume (Source → Channel → Sink, fan-in)**
log events must not be lost if the agent restarts	**Flume File channel**



Sqoop bulk-copies database tables to HDFS through parallel map tasks; Flume carries web-server logs to HDFS through an agent's Source, Channel and Sink.

Kafka: the append-only log

In one line

A Kafka offset is the integer position of a message inside a partition.

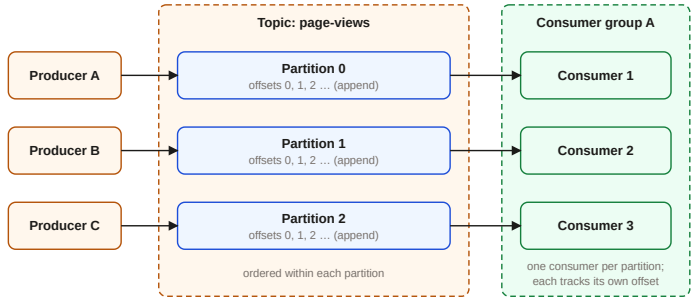
Traps

- Reading does not delete a message; retention decides deletion.
- Order is guaranteed only within a partition, not across a whole topic.

- Two different consumer groups each get every message; within one group a message goes to one consumer.
- The broker does not track consumer position; the consumer does.

Spot it in a scenario

If the question describes...	Answer
a new fraud model must be trained on last week's events without re-ingesting from the database	**retention + replay from offset 0**
fraud detection, balance update and audit all read every bKash transaction	**three consumer groups on one topic**
all pings from one driver must be processed in order	**key by driver_id: same key → same partition**
one server dies but no messages are lost	**replication factor > 1 (follower takes over as leader)**
a fifth consumer joins a group reading a 4-partition topic and gets nothing	**idle consumer: one partition per consumer per group**



Producers append to a topic's partitions; each partition is read by exactly one consumer in a group, which tracks its own offset.

Worked example

Slide: topic bkash-txns, P = 3 partitions, RF = 2. Copies = 3 × 2 = **6**; followers per partition = 1; survives **1** broker failure. Pathao driver-location: P = 10; a group of 4 gets 3, 3, 2, 2 partitions.

Step by step

1. Given P and R -> total partition copies = P × R
2. Given R -> followers per partition = R – 1
3. Given R -> broker failures survived = R – 1
4. Given P partitions and C consumers in one group -> active = min(P, C), idle = C – P if C > P
5. Given P = 10, C = 4 -> 10 / 4 = 2.5, so two consumers get 3 and two get 2

Variants

- page-views, 3 partitions, group of 5: 3 active, 2 idle
- P = 10, default R = 3: 30 copies, 20 followers, survives 2 failures
- P = 6, C = 4: two consumers get 2, two get 1

Why Kafka is fast

In one line

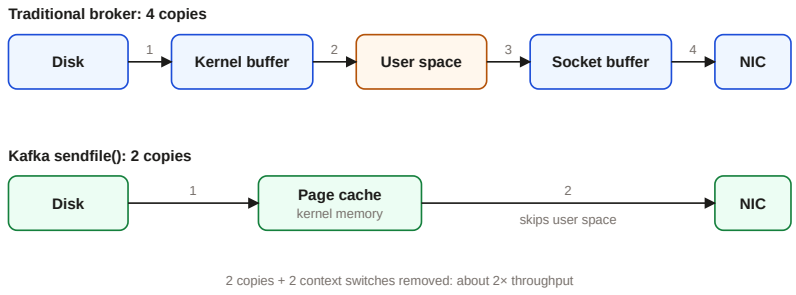
Zero-copy uses sendfile() to send data from disk to the network without copying it through user space.

Traps

- Zero-copy removes copies through user space; it does not mean no copies (2 remain).
- Two different random-disk numbers appear: < 1 MB/s (P1) and about 100 MB/s (paper). Quote the one the question gives.
- The benchmark table treats 1 MB as 1,000,000 bytes: 50.5 MB/s of 100-byte messages = 505,000 msgs/s.

Spot it in a scenario

If the question describes...	Answer
data goes from disk to network without passing through the application's memory	**zero-copy (sendfile())**
producer waits a few milliseconds to fill a bigger request	**batching (linger.ms, batch.size)**
Kafka runs on spinning disks yet nearly matches SSD speed	**sequential append-only writes + OS page cache**
throughput doubles when the topic goes from 4 to 8 partitions	**partition parallelism**



A traditional broker copies data four times through user space; Kafka's sendfile() zero-copy sends it from the page cache to the NIC in two.

Worked example

Paper, producer, 100-byte messages: 50.5 MB/s ÷ 100 B = 50,500,000 / 100 = **505,000 msgs/s**. Kafka vs ActiveMQ = 50.5 / 4.3 ≈ **11.7×**; vs RabbitMQ = 50.5 / 14.7 ≈ **3.4×**. Disk: 600 / 100 = **6×**. Zero-copy: 4 / 2 = **2×**.

Step by step

1. Given MB/s -> bytes/s = MB/s × 1,000,000
2. Given bytes/s and message size -> msgs/s = bytes/s ÷ size
3. Given two throughputs -> speedup = Kafka ÷ other
4. Given sequential and random MB/s -> advantage = sequential ÷ random
5. Given 4 copies (traditional) and 2 (sendfile) -> 2 copies saved per message, about 2×

Variants

- Consumer: 22.0 MB/s ÷ 100 B = 220,000 msgs/s; vs ActiveMQ 22.0 / 4.0 = 5.5×; vs RabbitMQ ≈ 2.7×
- 1,000-byte messages at 50.5 MB/s: 50,500 msgs/s
- Netflix: 100 billion events/day ÷ 86,400 ≈ 1,157,407 events/s average; if the 5× Super Bowl spike hit the 1.3 M/s peak, that would be 6.5 M/s
- Netflix 7-day retention holds 100 B × 7 = 700 billion events

Lambda and Kappa architectures

In one line

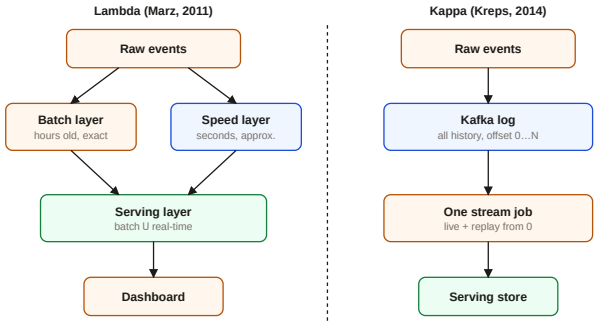
Kappa architecture drops Lambda's batch layer and reprocesses history by replaying the Kafka log.

Traps

- The Serving layer does not process data; real-time views come from the Speed layer (pool MCQ 3).
- Kappa is not 'Spark instead of Hadoop'; it removes the whole batch layer (pool MCQ 4, option d).
- There is no Ingestion layer in Lambda; that is a data-lake layer.
- Kappa keeps a serving store; what it drops is the batch layer.

Spot it in a scenario

If the question describes...	Answer
dashboard shows yesterday's totals from a nightly Hive job plus today's from Spark Streaming	**Lambda (batch + speed, merged in serving)**
a fee rule change must be coded in both the Hive job and the streaming job	**Lambda's dual code path weakness**
to fix a bug, a new stream job reads from offset 0 and then replaces the old one	**Kappa reprocessing**
the central bank requires a separate end-of-day batch reconciliation	**Kappa not suitable; keep Lambda**
the layer that only merges and answers queries	**Serving layer**



Lambda runs a batch and a speed layer merged by a serving layer; Kappa keeps one stream job over a replayable Kafka log.

Data lakes and the data swamp

In one line

A data swamp is a data lake with no governance: files without metadata, owners or quality checks.

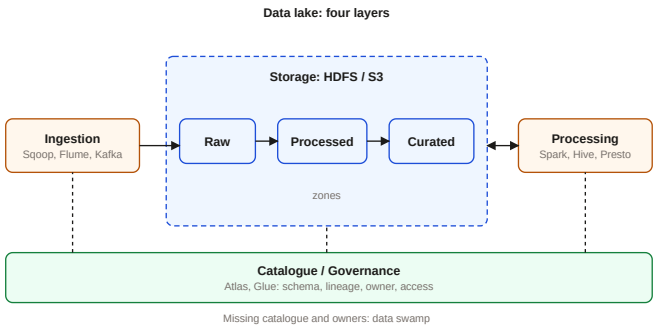
Traps

- A lake still needs a schema; it is applied on read, not removed.
- The swamp is caused by missing governance, not by data volume.
- Delta Lake and Iceberg are table formats that add ACID; they are not separate lakes.

Spot it in a scenario

If the question describes...	Answer
analysts cannot find datasets and nobody knows which copy is current	**data swamp**
images, JSON logs and CSV exports all stored as-is for later ML	**data lake (schema-on-read)**
finance needs a fixed monthly report from cleaned tables	**data warehouse**
Apache Atlas tracks lineage and owners for every dataset	**catalogue/governance layer**

If the question describes...	Answer
patient IDs are visible to every analyst in the lake	**swamp symptom (PII mixed); fix with access control**



A data lake's four layers: ingestion, zoned storage, processing, and the catalogue that keeps it from becoming a swamp.

Week 9

Data visualization

How to draw big data so people see the truth in it: Tufte's rules for clean, honest charts, Shneiderman's rule for interactive ones, and picking the chart from the question. Then the three problems scale adds (overplotting, latency, streaming), the tools that solve them, and the NYT COVID tracker as a worked case.

Exam questions

Multiple choice (4)

Q1 Edward Tufte's “data-ink ratio” principle states that a visualization should:

(a) Use more colours to highlight different categories of data

(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements

(c) Use ink sparingly to reduce printing costs

(d) Always include data labels using a different ink colour from the chart body

> **(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements**
> The ratio is about what the ink encodes, not saving ink.

Q2 Which visualization type is most appropriate for showing the distribution and spread of a continuous variable?

(a) Pie chart

(b) Line chart

(c) Histogram

(d) Treemap

> **(c) Histogram**
> Bins along the number line show shape and spread.

Q3 Shneiderman's visualization mantra begins with which step?

(a) Details first, then overview

(b) Overview first, zoom and filter, then details on demand

(c) Filter first, then provide an overview

(d) Details on demand first, then zoom out

> **(b) Overview first, zoom and filter, then details on demand**
> Filtering only makes sense after the overview shows what to filter.

Q4 Which visualization tool is best suited for a real-time production operations dashboard monitoring server metrics and application error rates with 5-second refresh intervals?

(a) Tableau

(b) D3.js

(c) Power BI

(d) Grafana

> **(d) Grafana**
> Native time-series DB integration with 5 s auto-refresh; D3 would need custom code.

True or false (4)

1 A pie chart is a recommended choice for comparing more than 5 categories.

Old slides (v1)

> **False.** People judge angles poorly. Corrected: use a horizontal bar chart; pies only for ≤ 4 categories.
>
> **From the lecturer’s files**
>
> **Model justification (old slides):** With more than 5 categories, people judge arc angles poorly; a horizontal bar chart sorted by value reads better.

2 Hexbin plots are an effective technique for reducing overplotting when visualising millions of data points.

> **True.** Counting points per hexagon turns 10 million dots into ≈ 10,000 coloured cells.

3 D3.js generates static PNG or PDF image files as its primary output format.

Old slides (v1)

False. Corrected: D3.js binds data to DOM elements and produces interactive SVG in the browser.

> **From the lecturer's files**
>
> **Model justification (old slides):** D3.js binds data to SVG DOM elements in the browser; the output is interactive, scalable vector graphics, not static image files.

4 Choropleth maps are used to visualise the geographic distribution of a quantitative variable.

Old slides (v1)

True. Regions are shaded by a value, best a rate such as cases per 100,000.

> **From the lecturer's files**
>
> **Model justification (old slides):** A choropleth encodes a quantitative variable (e.g. dengue incidence per 100,000) as colour intensity on geographic polygons.

Descriptive (3)

D1 Explain Tufte's data-ink ratio principle. Provide a "before and after" example: describe a cluttered bar chart (what chart-junk elements are present) and then describe the clean version after applying the principle.

Principle: data-ink ratio = ink encoding data ÷ total ink; maximise it. If erasing an element loses no information, erase it.

Before (quarterly sales)	After
Title "CHART TITLE!!!"	Plain short title
Legend for one series	No legend; Q1-Q4 on the x-axis
3-D bars with shadows	Flat 2-D bars, one colour
Gradient fill, border	Removed
Heavy dark gridlines	None, or one light line; y starts at 0

Same data, but almost all remaining ink is data, so the trend reads at once.

D2 What is overplotting? Describe three techniques to address it when visualising millions of data points in a scatterplot. For each technique, explain what information it preserves that the original overplotted chart lost.

Overplotting: millions of dots overlap into a blob that only says "data exists here".

Technique	How	Preserves
Alpha transparency	Each point α = 0.02; stacks look darker	Relative density (fails at very high density)
Hexbin	Count points per hexagon, colour by count; 10 M → ≈ 10,000 cells	Actual count per area
KDE contour	Smooth density, contour lines	Shape, modes (peaks), tails

D3 What is a data cube? How does pre-aggregating a billion-row fact table into a data cube enable interactive visualization at sub-second latency? What is the trade-off of this approach?

Data cube: pre-computed aggregates (SUM, COUNT) of a fact table for combinations of dimensions, e.g. district × date × product × channel.

- A nightly Spark/Hive job builds it once.
- It holds millions of rows, not billions.
- Tableau/Superset filters read the cube, so they answer in under 1 s instead of minutes.

Trade-off: staleness up to 24 h; only pre-built dimensions can be sliced; extra storage. Alternative: AQP (sampling) trades accuracy for speed.

Analytical (2)

A1 Public Health Dashboard Design: A DGHS public health dashboard needs to communicate: (a) COVID-19 confirmed case counts by district over 24 months (b) Real-time hospitalisation rate updated every 5 minutes (c) Demographic breakdown (age group, gender) of confirmed cases For each sub-requirement: specify the appropriate visualization type, justify using Shneiderman's data-type taxonomy, identify which big data visualization challenge applies (if any), and name the tool you would use. Apply Tufte's data-ink principle to state one specific chart-junk element to eliminate.

Req.	Data type	Chart	Challenge	Tool
(a) Cases by district, 24 months	Temporal (+ 2-D map)	Small multiples of line charts, same scale; choropleth of rate per 100,000	Latency: pre-aggregate to daily counts per district	Tableau / Power BI
(b) Hospitalisation every 5 min	Temporal (live)	Stat panel + sparkline, 24 h line chart	Streaming	Grafana on InfluxDB/Prometheus
(c) Age × gender	Categorical / multi-dimensional	Grouped horizontal bar	None (small table)	Tableau / Power BI

Sizes: (a) 730 days × 64 districts = **46,720** rows after aggregation. (b) 24 × 60 ÷ 5 = **288** points per day, so 5 s polling is ample.

Mantra: national total → district dropdown and date slider → hover tooltip.

Chartjunk to remove: 3-D effect on the demographic bars (also heavy gridlines on the line charts).

A2 Streaming Sentiment Visualization: Design a complete visualization system for a streaming Twitter (X) sentiment analysis pipeline processing 10,000 tweets per second. Specify: (a) which Kafka aggregation window you would use and why; (b) which chart types best convey sentiment drift over time and topic clustering; (c) how you would handle the velocity challenge at the visualization layer without overloading the browser; (d) which tools form the pipeline from Kafka to browser render.

Givens: 10,000 tweets/s → **600,000** per minute, **864,000,000** per day.

(a) 1-minute tumbling window per topic: count, mean sentiment, % positive/neutral/negative. One point replaces 600,000 tweets, smooths noise, still shows drift. Optional 15-min sliding window for a smoother trend.

(b) Drift: line chart (or 100% stacked area) per topic as small multiples. Clustering: t-SNE/UMAP scatter drawn as hexbin or KDE to avoid overplotting.

(c) Send only window aggregates (one refresh of 5 s replaces 50,000 tweets); keep a rolling buffer of the last 60 points; throttle redraws; pre-bin the cluster view server-side.

(d) X API → Kafka → Spark Structured Streaming/Flink → InfluxDB/Prometheus or Elasticsearch → Grafana (5 s poll) or Kibana; WebSocket → D3.js for sub-second views.

Discussion (7)

Lesson 1 Data quality is a visualisation problem: the NYT's biggest engineering effort was cleaning data, not drawing charts; retroactive state corrections forced D3 charts to support negative bars (days where cases were revised down)

See the diagram on the topic “Case study: NYT COVID-19 tracker”

Claim: a chart is only as right as its data.

For: most NYT work was normalising 50 state schemas; downward revisions made daily changes negative, so chart design had to show negative bars.

Against: cleaning is really a data-engineering task, and no chart design can fix bad source data. Even so, the chart must show corrections honestly (negative bars) rather than hide them.

Lesson 2 Accessibility is non-negotiable at 6M readers/day: every chart had a data table alternative for screen readers; colour scales were tested for colour-blindness (deuteranopia, protanopia)

Claim: at 6 M readers a day, even a small share of blind or colour-blind readers is many people.

For: a screen reader cannot describe the shapes of a chart, so a data table gives blind readers the same facts; avoiding red/green scales serves red-green colour-blind readers of public-health data.

Against: tables and testing cost time on daily deadlines and restrict palettes; the NYT judged it worth paying.

Lesson 3 Small multiples beat animation: the 50-state grid (Tufte small multiples) was more actionable than an animated map of spreading cases

Claim: showing every state at once beats one frame at a time.

For: a shared scale lets readers compare states side by side and find their own without waiting or remembering frames.

Against: animation grabs attention and shows geographic spread vividly; 50 small charts can be dense on a phone. Grid for analysis, animation for a headline.

Lesson 4 Rolling averages are editorial decisions: choosing 7-day vs. 14-day average changes what story the chart tells; these are journalistic choices with ethical weight

Claim: the window length shapes the story readers see.

For 7-day: removes weekend reporting dips and reacts to a real turn within days.

For 14-day: smoother, fewer false alarms, but hides a turning point longer.

Readers made health decisions from it, so the choice should be stated on the chart.

Lesson 5 GitHub as data infrastructure: publishing the raw CSV on GitHub enabled 1,000+ derivative dashboards worldwide within days

Claim: open data multiplies the value of expensive cleaning.

For: others reused it for 1,000+ dashboards; version history makes corrections traceable and numbers checkable.

Against: GitHub is not a database; others can misread the data, and the publisher must keep files consistent as corrections arrive.

v1-1 A Grafana dashboard shows daily API error counts for a Big Data pipeline. Identify three Tufte violations in a dashboard that uses: a 3-D bar chart with a y-axis starting at 500, a heavy grid, and gradient fill colours. Redesign the chart applying data-ink principles. [CLO3 — Apply]

Old slides (v1) CLO3

Violation	Principle broken	Fix
3-D bars	Chartjunk; depth and perspective distort bar heights	Flat 2-D bars (or a line, since daily counts are temporal)
y-axis starts at 500	Lie factor above 1: small differences look huge	Start y at 0
Heavy grid	Chartjunk competing with the data	No grid, or one light reference line (e.g. the error budget)
Gradient fill	Chartjunk: colour encodes nothing	One flat colour

Example: days with 600 and 700 errors are drawn 100 and 200 units tall (+100%) for a real +16.7%: lie factor = 100 ÷ 16.7 ≈ **6**.

Redesign: flat bars or line from 0, light axes, no legend for one series, label the axis directly, hover tooltip with the exact count (details on demand).

From the lecturer's files

> This discussion question was in the old (v1) slides' Part 2 only; the current deck keeps just the lessons.

v1-3 Design a Shneiderman-compliant interactive dashboard for a ride-sharing company's trip data (1M trips/day, 5 cities). Specify the overview view, the zoom/filter interactions, and the details-on-demand panel. Justify each design choice using the mantra. [CLO3 — Design]

Old slides (v1) CLO3

Givens: 1,000,000 ÷ 86,400 ≈ **11.6 trips/s**; about 200,000 trips per city per day.

Stage	Design	Why (mantra)
Overview	KPI row (trips today, completion rate, active drivers) with sparklines; 5-city small multiples of trips per hour on one scale; pickup map as a hexbin (1 M points would overplot)	The whole picture first, so the user sees which city or hour needs attention
Zoom and filter	City selector, date-range slider, hour brush, map zoom to a zone; served from a pre-aggregated cube (city × zone × hour) so each change returns in under 1 s	Narrow to the subset of interest only after seeing the overview; dynamic queries must feel instant
Details on demand	Hover a hex or bar for count, average wait and fare; click a zone or driver for a table of individual trips	Raw records only when asked, so the screen is not swamped

Live numbers: Kafka → Spark 1-min windows → time-series DB → Grafana 5 s refresh for the KPI row.

> **From the lecturer's files**
>
> This discussion question was in the old (v1) slides' Part 2 only; the current deck keeps just the lessons.

Live exercises (1)

Task Live Exercise — D3.js Bar Chart with Tufte Styling Task (25 minutes, pairs) Build a minimal D3.js bar chart that follows Tufte's data-ink principles: 1. No gridlines (or only a single light reference line) 2. y-axis starts at 0 3. No fill gradient — use a flat colour 4. Tooltip on hover (Shneiderman: details-on-demand) 5. Add a sparkline for each bar showing the 7-day trend Dataset: weekly Kafka message counts for 5 topics (generate 7×5 random values). Extension: add a dropdown filter to select one topic and zoom into its 7-day line chart.

```
<!DOCTYPE html>
<meta charset="utf-8">
<style>
  .bar { fill: steelblue; }
  .bar:hover { fill: #b5651d; }
  .axis path, .axis line {
    stroke: #ccc; stroke-width: 0.5px; }
  .tooltip {
    position: absolute; background: #fff;
    border: 1px solid #ccc; padding: 6px;
    font-size: 11px; pointer-events: none; }
</style>
<body>
<script src=
  "https://d3js.org/d3.v7.min.js"></script>
<script>
const topics = ["orders","clicks",
                "errors","logins","search"];
const data = topics.map(t => ({
  topic: t,
  count: Math.floor(Math.random()*50000)
         + 10000
}));
const svg = d3.select("body").append("svg")
  .attr("width", 500).attr("height", 300);
const x = d3.scaleBand()
  .domain(data.map(d=>d.topic))
  .range([50,480]).padding(0.3);
const y = d3.scaleLinear()
  .domain([0, d3.max(data,d=>d.count)])
  .range([260, 20]);
svg.append("g").attr("transform","translate(0,260)")
  .call(d3.axisBottom(x));
svg.append("g").attr("transform","translate(50,0)")
  .call(d3.axisLeft(y).ticks(4));
svg.selectAll(".bar").data(data).join("rect")
  .attr("class","bar")
  .attr("x",d=>x(d.topic))
  .attr("y",d=>y(d.count))
  .attr("width",x.bandwidth())
  .attr("height",d=>260-y(d.count));
</script>
```

Key code (added to the skeleton; 7 daily values per topic, bar = weekly sum; SVG height raised to 340 for the sparklines; CSS `.ref { stroke: #ddd; stroke-dasharray: 3 3; }`):

```
const data = topics.map(t => { const days = d3.range(7)
  .map(() => Math.floor(Math.random()*50000) + 10000);
  return {topic: t, days, count: d3.sum(days)}; });
svg.selectAll("g").attr("class","axis");   // skeleton forgot this
const mean = d3.mean(data, d => d.count);  // one light line at the mean
svg.append("line").attr("class","ref").attr("x1",50).attr("x2",480).attr("y1",y(mean)).attr("y2",y(mean));
const tip = d3.select("body").append("div").attr("class","tooltip").style("opacity",0);
svg.selectAll(".bar").on("mouseover",(e,d)=>tip.style("opacity",1)
      .style("left",(e.pageX+8)+"px").style("top",(e.pageY-28)+"px")
      .html(d.topic+": "+d.count))
    .on("mouseout",()=>tip.style("opacity",0));
const sx = d3.scaleLinear().domain([0,6]).range([0,x.bandwidth()]);
data.forEach(d => { const sy = d3.scaleLinear()
  .domain(d3.extent(d.days)).range([325,290]);
```

```
svg.append("path").attr("fill","none").attr("stroke","#555").attr("transform",`translate(${x(d.topic)},0)`)
 .attr("d", d3.line().x((v,i)=>sx(i)).y(v=>sy(v))(d.days)); });
```

1. **Gridlines:** D3 axes draw none; add only one dashed light line at the mean.
2. **y from 0:** domain `[0, max]` ; a higher start would push the lie factor above 1.
3. **Flat colour:** `fill: steelblue` , no gradient.
4. **Tooltip:** mouseover shows topic and count; mouseout hides it. This is details on demand.
5. **Sparkline:** 7-point line, no axes, under each bar, own min-max scale (shape, not size).

Extension: a `select` of the 5 topics; on change, fade other bars and draw that topic's 7-day line chart below.

Expected: 5 flat bars; tallest reaches y = 20 (240 px); bar width ≈ 56.8 px (step 430 ÷ 5.3 ≈ 81.1); weekly totals between 70,000 and 419,993.

Topics

Tufte: data-ink, chartjunk, lie factor

In one line

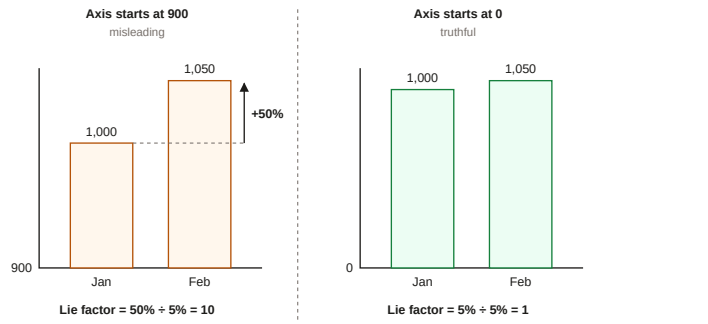
The data-ink ratio is the share of a chart's ink that encodes data, and Tufte says to maximise it.

Traps

- Data-ink ratio is not about saving ink or printing cost (MCQ option (c)).
- Lie factor uses percentage changes, not raw heights: divide the two changes, not the two bars.
- Small multiples need the same scale on every panel, or cross-panel comparison is invalid.
- Lie factor below 1 is also a distortion (understating), not "extra honest".

Spot it in a scenario

If the question describes...	Answer
a chart has 3-D bars, gradient fill, and a legend for one series	**chartjunk; low data-ink ratio**
the y-axis starts at 50, so a 10% rise looks like a doubling	**lie factor ≈ 10**
the same line chart drawn for every district in a grid, same y-scale	**small multiples**
a table of district case counts with a tiny trend line beside each number	**sparklines**
a dashboard designer removes gridlines and leaves one light line at the previous peak	**raising the data-ink ratio**



A y-axis starting at 900 makes a 5% rise (1,000 → 1,050) look like a 50% jump, lie factor 10; starting at 0 keeps it at 1.

Worked example

Lie factor (P2 slide 7). y-axis starts at 50 instead of 0. Data rises +10%, but the bar doubles in height (+100%). Lie factor = 100 ÷ 10 = **10**.

Data-ink ratio. 1,000 units of ink, 300 draw bars: 300 ÷ 1,000 = **0.3**. Remove 500 units of junk: 300 ÷ 500 = **0.6**.

Step by step

1. Given old and new data values → effect in data = (new − old) ÷ old
2. Given old and new drawn heights → effect shown = (new height − old height) ÷ old height
3. Given both effects → lie factor = shown ÷ data; compare with 1
4. Given data-ink and total ink → ratio = data-ink ÷ total ink
5. Given ink removed as junk → new total = old total − removed; data-ink unchanged; recompute ratio

Variants

- Revenue 100 → 110 (+10%); bars drawn 40 px → 60 px (+50%). Lie factor = 50 ÷ 10 = **5** (overstates)
- 400 data-ink + 600 decoration: ratio = 400 ÷ 1,000 = **0.4**; delete all decoration: 400 ÷ 400 = **1.0**
- Data +20%, drawn +10%: lie factor = 10 ÷ 20 = **0.5** (understates)
- Lecturer's textbook (ch 9): revenue 82M → 90M (+8 ÷ 82 = 9.76%); the y-axis starts at 80M, so the bars are 2 and 10 units tall: 2023 looks 5× as tall (+400%). Lie factor = 400 ÷ 9.76 ≈ **41**.

Shneiderman's mantra and choosing the chart

In one line

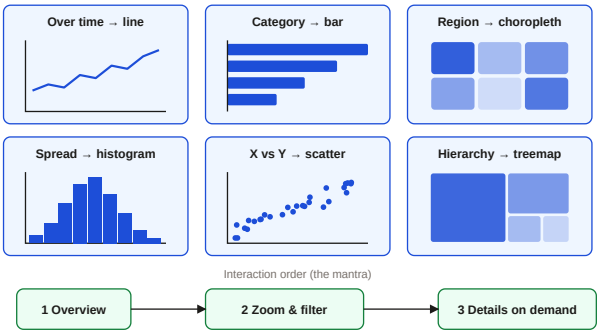
Shneiderman's mantra is "overview first, zoom and filter, then details on demand".

Traps

- Overview comes first. "Filter first" is the classic wrong option.
- A pie chart for more than 5 categories is never recommended (T/F trap).
- A histogram shows spread; a treemap shows hierarchy, not spread.
- The same r = 0.816 does not mean the same relationship (Anscombe).
- Seven data types (1-D … network) and seven tasks (overview … extract) are different lists; do not mix them.

Spot it in a scenario

If the question describes…	Answer
the dashboard opens on a national total, then a state dropdown, then a hover tooltip	**Shneiderman's mantra**
a date-range slider redraws the chart as it moves	**dynamic query (zoom and filter)**
disk usage by department drawn as nested rectangles sized by storage	**treemap (tree data)**
show how monthly cholera cases changed over three years	**line chart (temporal)**
show dengue cases per 100,000 people by district	**choropleth map (2-D)**



Pick the chart from the question and data type, then build the interaction in the mantra's order: overview, zoom and filter, details on demand.

Overplotting: alpha, hexbin, KDE

In one line

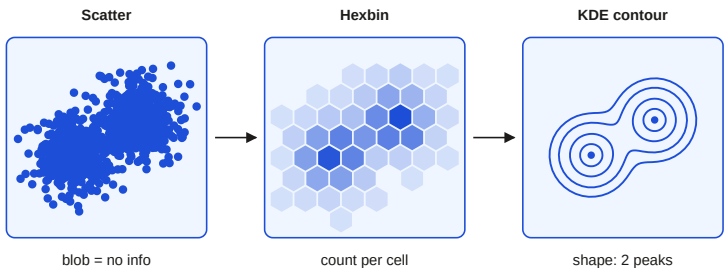
Overplotting is when millions of scatter points overlap into a solid blob that shows nothing.

Traps

- Alpha is the simplest fix but still fails at very high density; do not call it the best.
- Hexbin shows counts, KDE shows shape; do not swap what each preserves.
- Hexbins are True for reducing overplotting (T/F).

Spot it in a scenario

If the question describes...	Answer
a scatter of 10 million transactions is a solid colour	**overplotting**
the analyst needs the count of points in each region of the plot	**hexbin**
the analyst needs to see two peaks in the data	**KDE density contour**
points made 2% opaque, but the centre is still uniformly black	**alpha saturation; switch to hexbin or KDE**



Same 1,500 points drawn three ways

The same points as an overplotted scatter (a blob), a hexbin (count per hexagon) and KDE contours (two peaks revealed).

Worked example

Hexbin (P1 slide 17): 10,000,000 points → ≈ 10,000 hex cells. Reduction = 10,000,000 ÷ 10,000 = **1,000×** fewer shapes.

Alpha saturation (standard compositing, explains "fails at very high densities"): opacity after n stacked points = $1 - (1 - \alpha)^n$. At $\alpha = 0.02$, n = 100 gives **0.867**; n = 200 gives **0.982**.

Step by step

1. Given points and cells → reduction = points ÷ cells
2. Given α → single-point transparency = 1 − α
3. Given n stacked points → combined opacity = $1 - (1 - \alpha)^n$
4. Given target opacity p → n = ln(1 − p) ÷ ln(1 − α)

Variants

- 2,000,000 GPS pings into 500 hexes: 2,000,000 ÷ 500 = **4,000×**
- α = 0.02, n = 10: $1 - 0.98^{10}$ = **0.183**
- Points for 50% opacity at α = 0.02: ln 0.5 ÷ ln 0.98 ≈ 34.3, so **35** points

Latency: data cubes and AQP

In one line

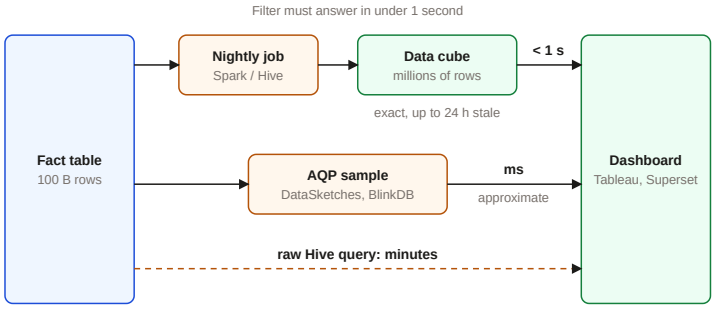
A data cube is a set of pre-computed aggregate tables over combinations of dimensions, read instead of the raw fact table.

Traps

- The cube's trade-off is staleness (and fixed dimensions), not wrong answers; AQP's trade-off is accuracy.
- The cube is built in batch ahead of time, not when the user clicks.
- Finest cube size is an upper bound: only combinations that occur are stored.

Spot it in a scenario

If the question describes...	Answer
a Tableau filter on a 100-billion-row Hive table takes minutes	**latency challenge**
a nightly job writes sales totals per district per day per product	**data cube (pre-aggregation)**
the dashboard shows yesterday's numbers in the morning	**cube staleness trade-off**
a dashboard returns an estimate in milliseconds from a sample	**AQP (DataSketches, BlinkDB)**



A nightly job pre-aggregates the fact table into a cube read in under 1 s (exact but up to 24 h stale); AQP answers from a sample in milliseconds (approximate); a raw Hive query takes minutes.

Worked example

Slide: 100 billion fact rows; cube of millions of rows. Taking 5 million: 100,000,000,000 ÷ 5,000,000 = **20,000×** fewer rows to read; the cube is **99.995%** smaller.

Step by step

1. Given distinct values per dimension → finest cube rows ≤ $d_1 \times d_2 \times \ldots \times d_k$
2. Given fact rows and cube rows → reduction = fact rows ÷ cube rows

3. Given k dimensions → group-by combinations = 2^k (including the grand total)

Variants

- 365 dates × 64 districts × 50 products × 4 channels = **4,672,000** rows at most
- 100 billion ÷ 4,672,000 ≈ **21,404×** fewer rows
- k = 4 dimensions → 2^4 = **16** group-by combinations
- Lecturer's textbook (ch 9): 120 B meter readings, 4 dimensions (meter_id 10 M, hour 43,800, region 50, tariff 5) → 2^4 = **16** cuboids. Hour × region cuboid = 43,800 × 50 = 2,190,000 rows × 30 bytes ≈ **65.7 MB**. Skip cuboids with meter_id: 10 M × 43,800 cells is more than the fact table itself.

Streaming dashboards and tools

In one line

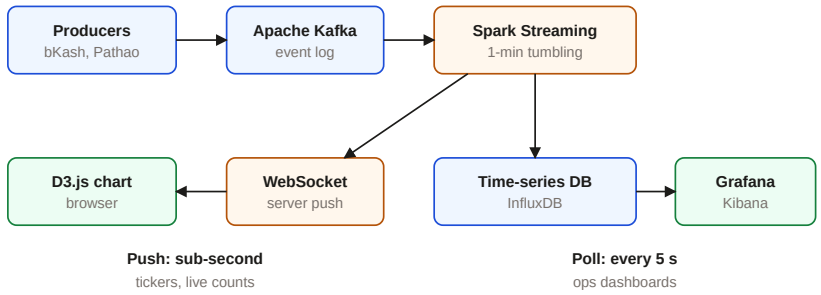
Grafana is the tool for real-time ops dashboards that poll a time-series database every few seconds.

Traps

- Tumbling windows do not overlap (each event counted once); sliding windows overlap.
- D3 can do real time, but the 5 s ops dashboard answer is Grafana: native time-series support, no custom code.
- D3.js output is SVG in the DOM, not static images (T/F).
- Send aggregates, never raw events, to the browser.

Spot it in a scenario

If the question describes...	Answer
server metrics and error rates, refresh every 5 seconds	**Grafana**
live election count updating many times a second in a news page	**WebSocket push to D3.js**
a manager wants drag-and-drop monthly sales reports	**Tableau or Power BI**
searching application logs stored in Elasticsearch	**Kibana**
open-source BI over Hive tables on a Hadoop data lake	**Apache Superset**



Kafka events are window-aggregated by Spark Streaming, then either stored in a time-series DB that Grafana/Kibana polls every 5 s, or pushed over a WebSocket to a D3.js chart for sub-second updates.

Worked example

Pool A2: 10,000 tweets/s with a 1-minute tumbling window → 10,000 × 60 = **600,000** tweets summarised into one point per topic. Per 5 s refresh: **50,000** tweets. Per day: 10,000 × 86,400 = **864,000,000**.

Step by step

1. Given event rate and window length → events per window = rate × seconds
2. Given refresh interval → events between refreshes = rate × interval
3. Given update interval → updates per day = 86,400 s ÷ interval

Variants

- Update every 5 minutes (A1(b)): 24 × 60 ÷ 5 = **288** points per day
- 1,000 events/s, 1-min window: **60,000** events per point

Case study: NYT COVID-19 tracker

In one line

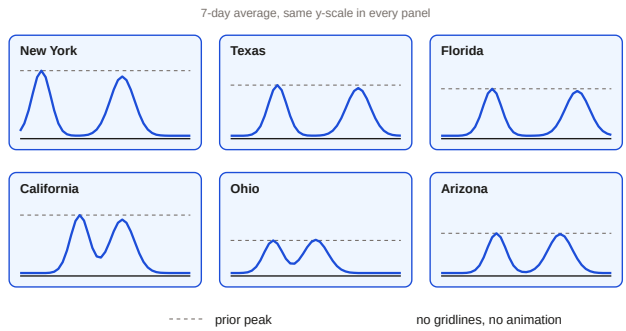
The NYT COVID tracker showed that cleaning data from 50 sources was harder than drawing the charts.

Traps

- The lesson is that cleaning, not rendering, was the biggest effort.
- A rolling average is an editorial choice, not a neutral fix.
- Deuteranopia and protanopia are red-green colour-blindness: avoid red/green scales.

Spot it in a scenario

If the question describes...	Answer
a state revises yesterday's cases down, so today's change is negative	**data quality shows up as a design problem (negative bars)**
every chart has a data table for screen readers	**accessibility lesson**
a grid of 50 state charts replaces an animated map	**small multiples beat animation**
outside groups build 1,000+ dashboards from a published CSV	**GitHub as data infrastructure**



Illustrative NYT-style small multiples: one 7-day-average chart per state on a shared y-scale, no gridlines, one light reference line at the prior peak.

Week 10

Big data in real applications

This chapter applies the course's stack (Kafka, Spark, HDFS, Hive, Grafana) to three domains: social media graphs, time series forecasting and healthcare data. A flu surveillance case study shows what big data signals can and cannot replace.

Exam questions

Multiple choice (4)

Q1 Facebook's Prophet library is primarily designed for:

(a) Graph analytics and community detection in social networks

(b) Real-time stream processing of events

(c) Time series forecasting with decomposable trend, seasonality, and holiday components

(d) Text sentiment classification using deep learning

★ Very likely | Lecturer's plan | CLO4

> **(c) Time series forecasting with decomposable trend, seasonality, and holiday components**
> Prophet fits $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$.

Q2 Which algorithm is most commonly used for community detection in large-scale social network graphs?

(a) K-Means clustering

(b) PageRank

(c) Louvain algorithm

(d) ARIMA

★ Very likely | Lecturer's plan | CLO4

> **(c) Louvain algorithm**
> It maximises modularity in O(n log n); PageRank ranks importance, not groups.

Q3 HIPAA in the United States primarily regulates the privacy and security of:

(a) Financial transaction data and bank records

(b) Healthcare data including patient records and medical information

(c) Social media user data and behavioural profiles

(d) Government data retention and public records

★ Very likely | Lecturer's plan | CLO4

> **(b) Healthcare data including patient records and medical information**
> HIPAA is the Health Insurance Portability and Accountability Act; financial data falls under GLBA.

Q4 DeepMind's AlphaFold used big data and deep learning to solve:

(a) Real-time financial fraud detection at bank-card transaction scale

(b) Social network influence maximisation over billion-node graphs

(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence

(d) Time series anomaly detection in IoT sensor streams

★ Very likely | Lecturer's plan | CLO4

> **(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence**
> Trained on 170,000 PDB structures with MSAs from 250M sequences; 2024 Nobel Prize in Chemistry.

True or false (4)

1 Google Flu Trends successfully replaced traditional CDC flu surveillance and has been used as the primary source ever since.

Old slides (v1)

> **False.** GFT over-predicted flu by 2× in 2012-2013 and was discontinued.
> **Corrected:** Google Flu Trends failed, and CDC still relies on clinical reporting, with big data signals only as a supplement.

> **From the lecturer's files**
>
> **Model justification (old slides):** Google Flu Trends over-predicted flu by about 2× in 2013 (over-fitted to search terms) and was dropped as a primary source; it only supplements traditional surveillance.

2 Time series data is order-sensitive: randomly shuffling the records in a time series changes the analysis result.

True. Autocorrelation, trend and seasonality exist only in time order; shuffling destroys them.

3 Social network graphs are typically sparse, with most nodes connected to only a small fraction of all possible neighbours.
Old slides (v1)

True. Real social networks are scale-free (power-law degrees): most nodes have few links and a few hubs have millions. Twitter had 1.47B edges among 41.7M users, about 35 follows each out of millions possible.

> **From the lecturer's files**
>
> **Model justification (old slides):** Real social networks are scale-free (power-law degrees): most nodes have few links while a few hubs have millions. So the average degree (about 35 follows among 41.7 M users) is tiny: the graph is sparse.

4 Medical imaging data such as MRI and CT scans is classified as structured data because it is stored in files.
Old slides (v1)

False. Being in a file does not make data structured.
Corrected: MRI and CT scans are unstructured binary DICOM files (200 MB to 2 GB), analysed by deep learning, not SQL.

> **From the lecturer's files**
>
> **Model justification (old slides):** MRI/CT images are unstructured binary DICOM files with no row-column schema; they need computer-vision models, not SQL.

Descriptive (3)

D1 Describe three types of analysis commonly performed on social media data (e.g., community detection, sentiment analysis, influence maximisation). For each, state one specific ethical risk that arises when the analysis is deployed at scale.

Analysis	What it does	Ethical risk at scale
Community detection	Louvain finds groups more densely linked inside than outside, by maximising modularity.	Profiling: group labels expose political or religious ties users never shared.
Sentiment analysis	VADER scores each post in [−1, +1]; > 0.05 positive, < −0.05 negative.	Suppressing dissent: flagging people who criticise a government policy.
Influence maximisation	Picks k seed users for widest spread; greedy gives ≥ 63% of optimal.	Manipulation: spreading misinformation or microtargeting vulnerable users.

Safeguard for all three: transparency and purpose limitation.

D2 What is the difference between ARIMA and Facebook Prophet for time series forecasting? Under what conditions would you prefer Prophet over ARIMA? When might ARIMA still be the better choice?

ARIMA(p, d, q) predicts from past p values and past q errors after differencing d times to make the series stationary. **Prophet** adds up trend with changepoints, Fourier seasonality and a holiday term: $y = g + s + h + \varepsilon$.

Prefer Prophet when the series has several seasonalities, holiday effects (Eid, Puja), trend shifts, missing data or outliers, or when thousands of series must be fitted automatically (5,000 SKUs in 12 minutes with applyInPandas).

ARIMA is still better for a short, stationary series with no strong seasonality and little data, where an expert can tune one important series closely.

D3 Identify three data integration and privacy challenges specific to healthcare big data analytics. For each, describe one technical mitigation strategy (e.g., differential privacy, federated learning, de-identification).

Challenge	Mitigation
Records carry names, DOB, NID, phone; HIPAA bars research use as-is	**De-identification:** safe harbour removes the 18 identifiers, or expert determination certifies low risk
Models need 1M+ records, but data cannot leave each hospital (consent, PDPA)	**Federated learning:** train at each hospital, share only model updates
Published counts for small areas can reveal one patient	**Differential privacy:** add calibrated noise to aggregates

Integration itself: FHIR, Parquet genomics, Kafka wearables and DICOM images meet in one HDFS/S3 lake with a Hive Metastore.

Analytical (2)

A1 Flu Outbreak Early-Warning System for Bangladesh: Design a big data system to detect flu outbreaks in Bangladesh using Twitter/Facebook posts and pharmacy purchase records from major pharmacy chains. (a) How would you collect and ingest these two data sources? (Specify tools: Kafka, Sqoop, Flume, REST API) (b) What features would you extract from each source for a prediction model? (Name specific NLP features and pharmacy signal metrics) (c) What model would you use, and how would you validate it against ground-truth CDC/DGHS surveillance data? (d) What are the ethical risks of this system? How would you mitigate them?

★ Very likely | Lecturer's plan | CLO4

Design: sources → Kafka → Spark Streaming → alert engine + Grafana (the CDC pattern).

Part	Social posts	Pharmacy sales
(a) Ingest	REST APIs (X API v2, Facebook Pages), keyword filter in Bangla and English, producer writes to Kafka topic `social_raw`	Sqoop for nightly bulk import from the chains' databases; Flume ships POS logs to Kafka topic `pharmacy_sales`
(b) Features	Daily flu-keyword post count per district; share of symptom terms (fever, cough, 'jor'); first-person illness flag; VADER/Bangla sentiment; ratio to 4-week baseline	Daily paracetamol and cough-syrup units per district; week-on-week % change; z-score vs same week last year

(c) Model: Prophet per district on DGHS weekly ILI counts, yearly seasonality, Eid holidays, social and pharmacy features as extra regressors; run districts in parallel with applyInPandas. **Validate** with a rolling backtest against DGHS counts: error (MAE), lead time, hit rate, false alarms, and test on an unseen season (GFT had $r^2 = 0.97$ in-sample and still failed).

(d) Ethics: privacy (keep district counts only, drop user IDs, add differential-privacy noise); panic from false alarms (an officer confirms before any announcement); bias against rural and older people who post less (weight with pharmacy and DGHS data); misuse for watching individuals (purpose limitation, access logs, PDPA consent).

A2 Retail Demand Forecasting Pipeline: A large Bangladeshi supermarket chain needs 90-day demand forecasts for 5,000 store–product SKU combinations. Design the end-to-end big data pipeline: (a) data ingestion from POS systems; (b) storage and partitioning strategy in HDFS/S3; (c) forecasting model choice (ARIMA vs. Prophet vs. LSTM) and justification; (d) how you parallelise model fitting across 5,000 SKUs using Spark; (e) how forecasts are served to the inventory management dashboard. Identify the primary bottleneck in your pipeline.

★ Very likely | Lecturer's plan | CLO4

Stage	Design
(a) Ingest	Tills send sale events to Kafka topic `pos_sales`; Sqoop bulk-loads history; Spark aggregates to (sku_id, date, units)
(b) Storage	Raw Parquet on HDFS/S3 partitioned by date; daily aggregate Hive table partitioned by date, bucketed by sku_id; forecasts partitioned by run_date

(c) Model	**Prophet:** weekly and yearly seasonality, holidays (Eid, Puja, Pohela Boishakh), automatic changepoints, tolerates gaps. ARIMA needs hand tuning per series; LSTM needs much data, GPUs and is a black box
(d) Parallel fit	`daily.groupBy("sku_id").applyInPandas(fit, schema)`; each group goes to one task, Prophet fits inside the Pandas UDF
(e) Serve	Write Parquet to Hive/S3; load latest run into a serving table; dashboard (Superset/Grafana) shows yhat with bands and reorder alerts

Numbers: 8 h = 480 min sequential; 12 min on 20 executors; speedup = 480 / 12 = **40×**; 5.76 s per model; 250 models per executor. One at a time that is 250 × 5.76 = 1,440 s = 24 min, so the slide's 12 min implies about 2 models at once per executor (40 slots).

Bottleneck: model fitting (Prophet via Stan, the 8-hour step). Skewed SKUs with long histories finish last; cap history and add cores.

Discussion (3)

1 Kwak et al. found Twitter's average shortest path is 4.12 hops over 41.7M users. If a misinformation tweet is retweeted by a high-PageRank account, estimate how many users it reaches within 3 hops assuming an average out-degree of 45. What graph property makes this estimate an overestimate? [CLO4 — Apply]

★ Very likely | Old slides (v1) | CLO4

Estimate: $45 + 45^2 + 45^3 = 45 + 2{,}025 + 91{,}125 =$ **93,195 users** (0.22% of 41.7M).

For an overestimate: clustering. Followers overlap, so the tree counts the same people again; most viewers also do not retweet.

Against: a high-PageRank account may have over a million followers, so hop 1 alone can beat 45.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

2 A Prophet model for weekly hospital admissions has strong annual and weekly seasonality but misses a sudden COVID-19 wave onset. Identify which Prophet component should detect this changepoint and explain what prior hyperparameter controls the model's sensitivity to new changepoints. [CLO4 — Analyse]

★ Very likely | Old slides (v1) | CLO4

Component: the trend g(t). A wave is a structural break, not a repeating cycle or a listed holiday.

Hyperparameter: `changepoint_prior_scale`, the scale of the sparse prior on the slope changes δ. Default 0.05 keeps the trend stiff and smooths over the wave; raising it (e.g. 0.5) lets the trend bend.

Against raising it: the trend then chases noise and the forecast band widens, so tune on held-out weeks.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

3 Design a dengue fever surveillance system for Bangladesh using BioSense-style ED data, Twitter signals, and Prophet forecasting. Specify: data sources, pipeline architecture (Kafka + Spark), model inputs, and how you would validate the system before policy use. [CLO4 — Design]

★ Very likely | Old slides (v1) | CLO4

- **Sources:** ED visits (FHIR) as main signal, NS1 lab results, dengue tweets as leading signal, rainfall.
- **Pipeline:** Kafka topics → Spark Streaming de-identifies and aggregates per district → Parquet → nightly Prophet per district via applyInPandas → alerts + Grafana.
- **Inputs:** weekly dengue-like visits, yearly (monsoon) seasonality, Eid holidays, tweets and rainfall as regressors.
- **Validate:** backtest on past seasons and an unseen season; compare with and without Twitter.

For: earlier warning. **Against:** tweets skew urban and young, so ED data stays primary.

> **From the lecturer's files**
>
> This discussion question appeared in the old (v1) slides' Part 2 too, with the same CLO tag.

Live exercises (6)

Task Live Exercise — Prophet Flu Forecasting in Python. Task (25 minutes, pairs): Fit a Prophet model to synthetic weekly flu incidence data and produce a 12-week forecast. Install: pip install prophet matplotlib pandas

```
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
dates = pd.date_range("2021-01-04",
                      periods=156, freq="W")
# Annual flu seasonality + noise
ili = (5 + 3*np.sin(
          2*np.pi*np.arange(156)/52
          - np.pi/2)
       + rng.normal(0, 0.5, 156)).clip(0)
df = pd.DataFrame({"ds": dates, "y": ili})

# Optional: add Eid holiday
holidays = pd.DataFrame({
   "holiday": "Eid",
   "ds":       pd.to_datetime([
                 "2021-05-13","2022-05-03",
                 "2023-04-22"]),
   "lower_window": 0,
   "upper_window": 3,
})

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    changepoint_prior_scale=0.05,
    holidays=holidays)
m.fit(df)
future = m.make_future_dataframe(
    periods=12, freq="W")
forecast = m.predict(future)
m.plot(forecast)
m.plot_components(forecast)
plt.show()
```

Run the skeleton, then refit with the changes below.

```
m = Prophet(yearly_seasonality=True,
            weekly_seasonality=False,
            changepoint_prior_scale=0.05,   # try 0.5
            holidays=holidays,
            interval_width=0.95)            # 95% band
m.fit(df)
fc = m.predict(m.make_future_dataframe(periods=12, freq="W"))
m.plot(fc); m.plot_components(fc)
```

Expected: dates run 2021-01-10 to 2023-12-31 (weeks end Sunday); the forecast covers 2024-01-07 to 2024-03-24 and rises from the yearly low (true curve 2.0) towards about 4.3.

1 Generate 3 years of synthetic weekly ILI data with annual seasonality + noise.

> The skeleton does it: 156 weekly dates (156 / 52 = 3 years), a sine with period 52 weeks running from 2 to 8 around a mean of 5, plus noise with SD 0.5. Columns must be named `ds` and `y`.

2 Fit Prophet with yearly and weekly seasonality.

> `yearly_seasonality=True` fits the annual Fourier wave (P = 365.25). Keep `weekly_seasonality=False`: with one point per week, always a Sunday, there is no within-week pattern to learn.

3 Plot the forecast with 95% uncertainty intervals.

> Pass `interval_width=0.95` (default is 80%), then `m.plot(forecast)`. Expected: data dots, the yhat line and a band from yhat_lower to yhat_upper that widens into the future.

4 Add a holiday for “Eid” (spike in healthcare visits) and observe the effect.

> Expected effect is near zero: the synthetic data has no Eid spike, and the 2022 window (05-03 to 05-06) contains no Sunday, so it matches no point. Add +3 to the Eid weeks and the components plot shows an Eid bar near +3.

5 Change changepoint_prior_scale from 0.05 to 0.5 and compare trend flexibility.

> At 0.05 the trend stays nearly flat around 5 (the true trend). At 0.5 it wiggles, absorbs noise and part of the seasonal wave, and the future band widens. Higher reacts faster to real breaks but overfits stable data.

Topics

Social media as a graph: communities, influence, Twitter's shape

In one line

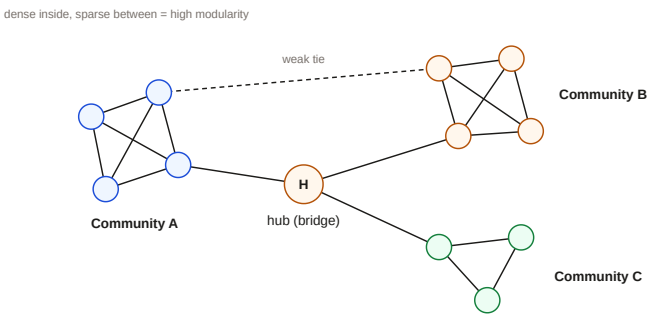
Community detection finds groups of nodes more densely linked to each other than to the rest of the graph, usually with the Louvain algorithm.

Traps

- PageRank measures importance, not community membership; K-Means clusters vectors, not graph links.
- The hop estimate is an upper bound: it counts shared followers again (clustering, overlapping neighbourhoods).
- Kempe's result: the problem is NP-hard, the greedy answer is approximate (63%), not exact.
- Sparse means few edges per node, not few nodes.

Spot it in a scenario

If the question describes...	Answer
find tightly linked groups of accounts, such as a coordinated bot network	**community detection (Louvain)**
a ministry wants to pick 50 accounts to seed a vaccine message so it reaches the most people	**influence maximisation (Kempe greedy)**
rank accounts by how important they are in the follow graph	**PageRank, not community detection**
most follows are one-way and trending topics are headline news	**Twitter as a news medium (Kwak 2010)**
estimate how many users a tweet reaches in h hops with out-degree d	**tree estimate d + d^2 + ... + d^h, an overestimate**



Communities are dense inside and sparsely linked between, which is what Louvain's modularity rewards; a hub bridges them.

Worked example

Discussion Q1 (P2 slide 15): out-degree d = 45, h = 3 hops, 41.7M users.

Hop	New users
1	45
2	45^2 = 2,025
3	45^3 = 91,125
Total	**93,195** (0.22% of 41.7M)

Step by step

1. Given d = 45 -> hop 1 reaches 45 users
2. Given hop 1 = 45 and d = 45 -> hop 2 = 45 × 45 = 2,025
3. Given hop 2 = 2,025 -> hop 3 = 2,025 × 45 = 91,125
4. Given hops 45, 2,025, 91,125 -> total = 93,195
5. Given total 93,195 and 41.7M users -> share = 93,195 / 41,700,000 = 0.22%

Variants

- d = 30, h = 3: 30 + 900 + 27,000 = 27,930 users.
- d = 40, h = 2: 40 + 1,600 = 1,640 users.
- d = 45, h = 4: adds 45^4 = 4,100,625, total 4,193,820.
- Greedy seeding, optimal reach 10,000: guaranteed at least 10,000 × (1 – 1/e) ≈ 6,321 users.
- Lecturer's textbook (ch 10): optimal seed set reaches 200,000 users. Greedy worst case = 200,000 × (1 – 1/e) = 200,000 × 0.632 ≈ 126,424 users.

Sentiment (VADER), trend detection and ethics

In one line

VADER is a lexicon-and-rule-based sentiment analyser for social media that outputs a compound score in [–1, +1].

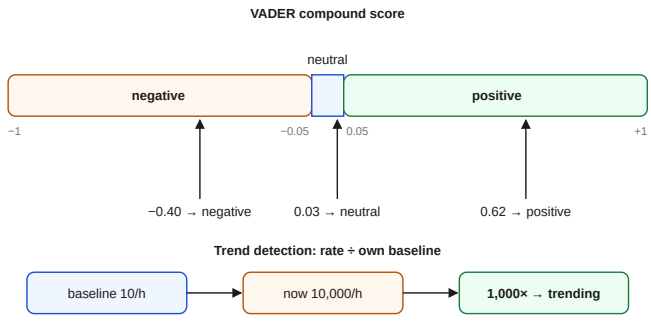
Traps

- The slide's cut-offs are strict (> 0.05, < –0.05), so 0.05 itself is neutral by the slide. VADER's own documentation uses ≥ 0.05 and ≤ –0.05, so if a boundary value is asked, state the slide's rule you are using.

- Trending depends on the ratio to baseline, not on absolute volume.
- VADER is rule-based, not a deep-learning classifier.
- LSH here is for detecting trends, not for sentiment.

Spot it in a scenario

If the question describes...	Answer
classify posts as positive, neutral or negative, including emojis and 'NOT good'	**VADER sentiment analysis**
a phrase normally seen 10 times an hour suddenly appears 10,000 times	**trend detection (spike against baseline, LSH on n-grams)**
a government flags accounts posting negative views on its policy	**ethical risk: suppressing dissent**
check whether a hashtag spike is real celebration or a bot campaign	**sentiment breakdown of the spike**



VADER splits the compound score at ±0.05, and trend detection divides a phrase's current rate by its own baseline.

Worked example

P1 slide 12: a phrase goes from 10/hour to 10,000/hour.

Spike ratio = current / baseline = 10,000 / 10 = **1,000×**, so it is trending.

VADER on three posts: 0.62, 0.03, −0.40 → positive, neutral, negative.

Step by step

1. Given baseline 10/hour and current 10,000/hour -> ratio = 10,000 / 10 = 1,000×
2. Given a compound of 0.62 -> 0.62 > 0.05, positive
3. Given a compound of 0.03 -> between −0.05 and 0.05, neutral
4. Given a compound of −0.40 -> < −0.05, negative

Variants

- #QurbaniEid spike 1,200× on a baseline of 50 posts/hour: peak = 50 × 1,200 = 60,000 posts/hour.
- 20/hour to 6,000/hour: 6,000 / 20 = 300×.
- Compound exactly 0.05: not greater than 0.05, so neutral by the slide's rule (VADER's own docs, using ≥ 0.05, would call it positive). −0.06: negative.

Time series, ARIMA and Prophet at scale

In one line

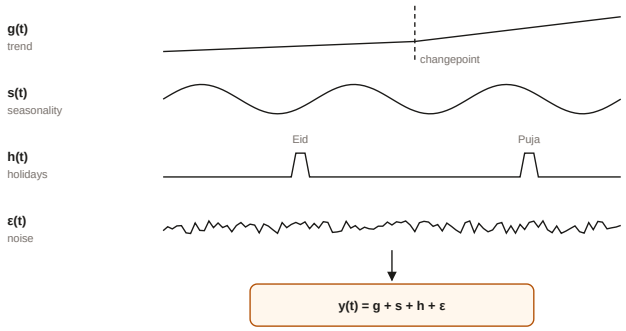
Prophet is a time series forecasting model that splits a series into trend, seasonality, holidays and noise: y(t) = g(t) + s(t) + h(t) + ε(t).

Traps

- A sudden wave is a trend changepoint, not seasonality; h(t) only covers dates you list in advance.
- Weekly seasonality on weekly data learns nothing: every point is the same weekday.
- Prophet's default interval is 80%; set interval_width=0.95 for 95%.
- ARIMA's d is the number of differences, not the number of seasons.

Spot it in a scenario

If the question describes...	Answer
forecast thousands of store-product series with Eid and Puja effects	**Prophet with a holiday list, parallel via applyInPandas**
one short, stable series with no strong seasonality	**ARIMA**
the model misses a sudden wave, a break in the trend	**trend g(t) changepoints; raise changepoint_prior_scale**
5,000 models take 8 hours on one machine	**Spark groupBy + applyInPandas (40× speedup)**
rows are shuffled before training and the pattern disappears	**order matters / autocorrelation**



Prophet adds four parts: a trend with changepoints, Fourier seasonality, listed holidays and noise.

Worked example

P1 slide 19: 5,000 SKUs, 8 hours sequential, about 12 minutes on 20 executors.

Quantity	Working	Result
Speedup	480 min / 12 min	**40×**
Time per model	28,800 s / 5,000	**5.76 s**
Models per executor	5,000 / 20	**250**

Check: one model at a time, an executor would need 250 × 5.76 s = 1,440 s = 24 min. The slide's 12 min is half that, so about 2 models run at once per executor (40 in total, e.g. 2 cores each). That is why 20 executors give 40×, not 20×. (Inference; the slide does not state cores.)

Prophet sum: Eid week with g = 100, s = +15, h = +30, ε = −2 gives y = **143**.

Step by step

1. Given 8 hours -> 8 × 60 = 480 minutes
2. Given 480 min sequential and 12 min parallel -> speedup = 480 / 12 = 40×
3. Given 8 h = 28,800 s and 5,000 models -> 5.76 s per model
4. Given 5,000 models and 20 executors -> 250 models per executor
5. Given g = 100, s = 15, h = 30, ε = −2 -> y = 143

Variants

- 10,000 SKUs at 5.76 s, 40 models at once: sequential 57,600 s = 16 h; parallel 1,440 s = 24 min; speedup still 40×.
- 400 wards at 2 s each, 20 in parallel: 800 s sequential, 40 s parallel, 20×.
- Non-holiday week, y = 120, g = 110, ε = 0: s = +10.
- Changepoint at t = 10: k = 0.5, m = 1, δ = 0.3, γ = −3. g(10) = 6 both sides, g(20) = 0.8 × 20 + 1 − 3 = 14.
- Lecturer's textbook (ch 10): 6,000 towers at 2 s per Prophet fit. Sequential 12,000 s; one 8-core machine 12,000 / 8 = 1,500 s = 25 min; 20 executors × 8 cores = 160 slots: 6,000 / 160 = 37.5, so 38 rounds × 2 s = 76 s.

Anomaly detection: STL and Isolation Forest

In one line

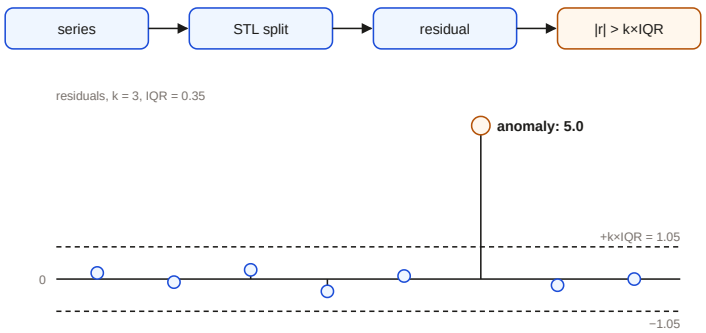
An anomaly is a data point that does not conform to the expected pattern of the series.

Traps

- Use the absolute value of the residual: big negative residuals are anomalies too.
- The threshold uses the IQR of residuals, not of the raw series.
- In Isolation Forest, anomalies have shorter paths, not longer.

Spot it in a scenario

If the question describes...	Answer
an agent's hourly transactions jump from 10 to 10,000	**Isolation Forest anomaly (possible money laundering)**
flag days whose value is far off after removing the weekly and yearly cycle	**STL residual with k × IQR**
a wearable streams heart rate per second and a sudden drop must raise an alert	**streaming anomaly detection on time series**



STL removes trend and seasonality, then flags any residual whose absolute value exceeds k × IQR (here 3 × 0.35 = 1.05).

Worked example

Residuals: 0.2, −0.1, 0.3, −0.4, 0.1, 5.0, −0.2, 0.0; k = 3.

1. Sorted: −0.4, −0.2, −0.1, 0.0, 0.1, 0.2, 0.3, 5.0
2. Q1 = −0.125, Q3 = 0.225 (inclusive interpolation)
3. IQR = 0.35; threshold = 3 × 0.35 = 1.05
4. Only |5.0| > 1.05: the 6th point is the **anomaly**

Step by step

1. Given the residuals -> sort them
2. Given the sorted list -> Q1 = −0.125, Q3 = 0.225
3. Given Q1 and Q3 -> IQR = 0.225 − (−0.125) = 0.35
4. Given IQR 0.35 and k = 3 -> threshold 1.05
5. Given threshold 1.05 -> compare each |residual|; only 5.0 exceeds it

Variants

- Same data, k = 1.5: threshold 0.525; largest normal |−0.4| = 0.4, so still only 5.0 is flagged.
- IQR = 2 and k = 3: threshold 6, a residual of −7 is flagged (absolute value).

Healthcare big data, privacy and AlphaFold

In one line

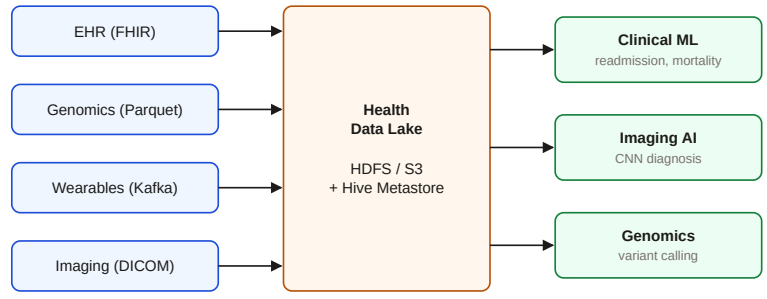
Federated learning trains a model at each hospital and shares only model updates, so raw patient records never leave the site.

Traps

- Stored in a file does not mean structured: DICOM images are unstructured.
- HIPAA is for health data; US financial data falls under GLBA.
- Federated learning moves the model, not the data; differential privacy adds noise, it does not encrypt.
- AlphaFold predicts structure from sequence; it is not anomaly or fraud detection.

Spot it in a scenario

If the question describes...	Answer
five hospitals want one sepsis model but cannot share patient files	**federated learning**
a health ministry publishes district counts of a rare disease	**differential privacy**
strip names, dates of birth, NIDs and phone numbers before research	**de-identification (safe harbour)**
predict a protein's 3D shape from its amino-acid sequence	**AlphaFold**
CT and MRI scans of 1 GB each	**unstructured DICOM imaging, analysed with CNN/ViT**



Four health data types land in one HDFS/S3 lake with a Hive Metastore and feed clinical, imaging and genomics models.

Worked example

P1 slide 21: 10,000 patients × 200 GB per genome = 2,000,000 GB = **2 PB** (1 PB = 1,000,000 GB).

Step by step

1. Given 200 GB per genome and 10,000 patients -> 2,000,000 GB
2. Given 2,000,000 GB -> divide by 1,000,000 = 2 PB
3. Given 2 PB and HDFS replication 3 -> 6 PB of raw disk

Variants

- 2,500 patients with replication 3: 2,500 × 200 = 500 TB; × 3 = 1.5 PB.
- 5,000 patients: 5,000 × 200 = 1,000,000 GB = 1 PB.

Syndromic surveillance and the Google Flu Trends lesson

In one line

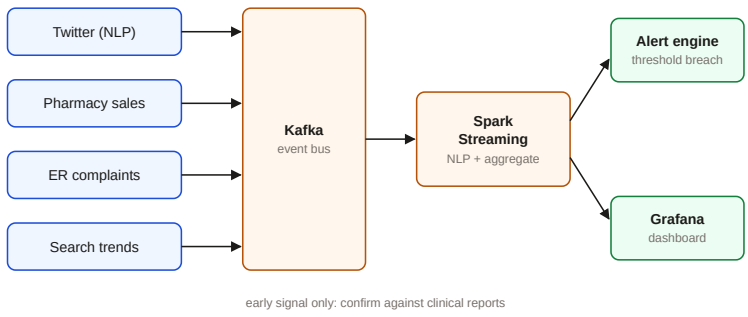
Syndromic surveillance uses non-traditional signals, such as posts, pharmacy sales and ER notes, that show illness before a diagnosis is reported.

Traps

- GFT did not replace CDC surveillance; it failed and was discontinued.
- Syndromic signals are early warnings, not ground truth.
- Twitter enters the post-GFT model as a covariate, not as the main signal.
- $r^2 = 0.97$ was in-sample; it says nothing about future seasons.

Spot it in a scenario

If the question describes...	Answer
paracetamol sales jump across a district before hospitals report cases	**syndromic surveillance (pharmacy signal, 3-5 days ahead)**
a search-based model fit past years well but doubled the real flu rate	**Google Flu Trends lesson: supplement, not replace**
media coverage drives searches up while cases stay low	**proxy data amplifies noise**
combine ED visits as the main input with tweets as an extra regressor	**post-GFT Prophet nowcasting with a covariate**



Syndromic surveillance streams four early signals through Kafka into Spark Streaming, which drives alerts and a Grafana dashboard.

Mock papers

Mock final paper

One 10-mark question per chapter, in the shape of the October 2026 Data Mining final from the same department: a short scenario, then short parts. Answer either the main pair or the OR pair. Tap a part for its model answer.

Question 1

Week 5, 10 marks

(a)

4 marks

A bank keeps 20 TB of transaction logs with 50 columns as gzip-compressed CSV on HDFS. A fraud report reads 5 columns.

i) Why does each file get only one Mapper?

ii) Which format and codec should replace it?

iii) How much data would the report read after the switch?

iv) What in the new format lets it skip whole blocks of rows?

Model answer

i) Gzip is not splittable, so reading must start at the file's beginning.

ii) Parquet + Snappy (or ORC for Hive).

iii) 20 TB × 5/50 = **2 TB**, a 90% cut.

iv) Min/max stats per column chunk in the footer: predicate pushdown.

(b)

6 marks

A cluster of 1,000 nodes, each 128 GB RAM and 32 vCPU, runs YARN with queues analytics 50%, streaming 30%, adhoc 20%. Analytics runs Spark executors {8 GB, 4 vCPU}.

i) Which YARN component grants the containers?

ii) Find the maximum executors in the analytics queue and the bottleneck.

iii) A node running 6 executors dies: who detects it and who asks for replacements?

iv) Which scheduler fits these three teams, and why?

Model answer

i) The ResourceManager's Scheduler.

ii) 500 nodes; RAM 64,000 GB / 8 = 8,000; vCPU 16,000 / 4 = 4,000; max **4,000**, CPU bottleneck.

iii) The RM detects missed heartbeats (>10 min) and marks the 6 containers failed; the Spark ApplicationMaster requests 6 replacements.

iv) Capacity: each team gets a guaranteed share and can borrow idle capacity.

OR

(a) 4 marks

A company runs a 5,000-node Hadoop 1.x cluster and wants to add Spark jobs.

i) Name two limits of the JobTracker.

ii) What did Hadoop 2.0 split it into?

iii) How many ApplicationMasters run for 12 active jobs?

iv) What is a container?

Model answer

i) MapReduce only; about 4,000-node ceiling (5,000 is past it).

ii) A global ResourceManager and a per-job ApplicationMaster.

iii) 12, one per job.

iv) A slice of one node: set vcores plus set RAM, in which a task runs.

(b) 6 marks

A ride-sharing app streams trip events through Kafka; the schema gains a field each month. Each night a Spark job builds reports; 10 GB of the day's data is written per night.

i) Which format at ingest, and why?

ii) What happens when a new reader opens an old file missing the new field?

iii) Which format and codec for the nightly analytics copy?

iv) How many Mappers can read the 10 GB night file with 128 MB row groups, and how many if it were one gzip CSV?

Model answer

i) Avro: row-oriented and append-friendly, with schema evolution.

ii) The missing field gets its default value from the reader schema.

iii) Parquet + Snappy.

iv) 10,240 / 128 = **80** Mappers; gzip CSV: **1**.

Question 2 Week 6, 10 marks

(a) 4 marks

A ride-sharing firm runs a PySpark job on its Hadoop cluster: `textFile → filter(fare > 0) → map(to (zone, fare)) → reduceByKey(sum) → saveAsTextFile`.

i) Which component builds the DAG and schedules tasks?

ii) How many stages run, and where is the boundary?

iii) Which call starts the computation, and why?

iv) Why is reduceByKey better here than groupByKey?

Model answer

i) The Driver (YARN is the Cluster Manager, Executors run the tasks).

ii) Two: textFile → filter → map are narrow and pipelined in Stage 1; reduceByKey is wide, so the shuffle starts Stage 2.

iii) saveAsTextFile, the only action; transformations are lazy.

iv) It sums fares per zone inside each partition before the shuffle, so far fewer records cross the network.

(b) 6 marks

The firm trains a demand model with 20 iterations over 40 GB. HDFS reads and writes at 400 MB/s; RAM cache reads at 4,000 MB/s.

i) Total HDFS I/O for MapReduce.

ii) Spark's HDFS I/O and the reduction factor.

iii) Wall-clock time for each.

iv) The speedup, and one case where Spark would not win.

Model answer

i) 20 × (40 + 40) = 1,600 GB = 1.6 TB.

ii) 40 GB (read once); 1,600 / 40 = 40× less.

iii) HDFS pass = 40,000 / 400 = 100 s, so T_MR = 20 × 200 = 4,000 s ≈ 66.7 min. RAM pass = 40,000 / 4,000 = 10 s, so T_Spark = 100 + 19 × 10 = 290 s ≈ 4.8 min.

iv) 4,000 / 290 ≈ **13.8×**. Spark would not win on a single-pass job, or if the 40 GB did not fit in cluster RAM.

OR

(a) 4 marks

A job runs `input → filter → map` over 8 partitions and caches the result. The Executor holding partition 4 crashes.

i) What does “resilient” mean for this RDD?

ii) List the recovery steps.

iii) What fraction of the work is redone?

iv) What changes if the next step is groupByKey?

Model answer

i) Lost partitions are recomputed from lineage, not read from a replica.

ii) Detect the lost partition → trace lineage to the input → re-read input partition 4 from HDFS → re-apply filter then map.

iii) 1 / 8 = 12.5%.

iv) groupByKey is wide: a lost output partition needs data from all 8 parent partitions, so recovery can rerun the whole earlier stage.

(b) 6 marks

A word count runs on 6 partitions. Each partition holds 2,000 `(word, 1)` pairs with 40 distinct words.

i) Records shuffled by groupByKey.

ii) Maximum records shuffled by reduceByKey.

iii) Percentage reduction.

iv) With numPartitions = 5, which partitions do keys with hash 23 and 40 go to, and is this step narrow or wide?

Model answer

i) 6 × 2,000 = 12,000.

ii) 6 × 40 = 240.

iii) 1 – 240 / 12,000 = 0.98 → **98%** fewer records.

iv) 23 % 5 = 3 and 40 % 5 = 0, so partitions 3 and 0. Hash partitioning for a shuffle is a wide dependency.

Question 3

Week 7, 10 marks

(a)

4 marks

A food-delivery company stores 4 TB of orders in one Hive table. Managers run a daily report filtered by order_date and city, and analysts join orders with a 15 M-row riders table on rider_id. Today every report scans the whole table.

i) Which feature lets the daily report read only one folder?

ii) Name the columns you would use for it.

iii) Why not use rider_id for that feature?

iv) What should be done to rider_id instead?

Model answer

i) Partitioning (partition pruning).

ii) order_date and city.

iii) High cardinality: millions of directories would overload the NameNode.

iv) Bucket both tables on rider_id with the same number of buckets for a bucket map join.

(b)

6 marks

For the table in (a):

i) The table is partitioned by 730 days and 20 cities. What fraction does a one-day, one-city report read?

ii) With 4 TB and 8 buckets per partition, what is the size of each bucket file?

iii) Which bucket holds rider_id 1,000,003?

iv) Write the CREATE TABLE line that declares the partitions and buckets.

Model answer

i) P = 730 × 20 = 14,600; 1 / 14,600 ≈ 0.0068%.

ii) 4,000 GB / (14,600 × 8) = 4,000 / 116,800 ≈ 34 MB (small; fewer buckets, e.g. 2, give ≈ 137 MB).

iii) 1,000,003 % 8 = 3.

iv) `PARTITIONED BY (order_date STRING, city STRING) CLUSTERED BY (rider_id) INTO 8 BUCKETS STORED AS ORC`.

OR

(a)

4 marks

A telecom's analysts send HiveQL from Tableau. A query `SELECT region, COUNT(*) FROM calls GROUP BY region` takes 9 minutes.

i) Which component receives the query?

ii) Where does Hive find the table's HDFS path?

iii) What does the map phase emit?

iv) Why is it slow on MapReduce?

Model answer

i) HiveServer2.

ii) The Metastore.

iii) (region, 1), or partial counts per region with map-side aggregation.

iv) MapReduce writes intermediate results to HDFS between stages.

(b) 6 marks

i) The calls table has 60 million rows and 300 regions; one mapper uses map-side aggregation. By what factor does the shuffle shrink?

ii) The team moves the dashboard to Presto and the query drops from 9 minutes to 27 seconds. Find the speed-up.

iii) Give two Presto design decisions behind it.

iv) Does the team need to redefine its tables for Presto? Why?

Model answer

i) 60,000,000 / 300 = 200,000×.

ii) 540 s / 27 s = 20×.

iii) No intermediate disk writes (in-memory pipeline); vectorized execution on 1,024-row batches (also: cost-based optimizer).

iv) No: Presto reads the same Hive Metastore.

Question 4 Week 8, 10 marks

(a) 4 marks

A ride-sharing firm streams driver GPS pings every 3 seconds to a Kafka topic with 10 partitions, keyed by driver_id, replication factor 3. The routing engine and the billing service both read the topic.

i) Why key by driver_id?

ii) How do routing and billing each get every ping?

iii) How many broker failures can the topic survive?

iv) Routing scales to 14 consumers in one group. How many sit idle?

Model answer

i) Same key → same partition, so each driver's pings stay in order.

ii) Each is its own consumer group.

iii) R − 1 = 2.

iv) 14 − 10 = 4 idle.

(b) 6 marks

The same firm loads its 5 M-row trips table from MySQL into HDFS nightly and also wants a fraud check on payments within 1 second. Name the tool for each, choose Lambda or Kappa for the whole system, and give one weakness of your choice.

Model answer

Trips table: Sqoop (incremental import by last-value, parallel map tasks, retry on failure). Payments: Kafka topic + Spark Streaming (milliseconds, RF for fault tolerance, replay from offset). Architecture: Lambda, because a real nightly batch path exists (batch = Sqoop + HDFS + Hive; speed = Kafka + Spark Streaming; serving merges). Weakness: two code paths for shared logic, 30–40% of effort to keep in sync; reduce it by making Kafka the single source both layers read.

OR

(a) 4 marks

A hospital stores five years of lab CSVs, scanned reports and device logs in S3 with no catalogue. Analysts find four versions of the same table, and patient IDs are open to every user.

i) Lake or warehouse?

ii) What has it become?

iii) Name the layer that is missing.

iv) Give two fixes.

Model answer

i) Data lake.

ii) A data swamp.

iii) Catalogue/Governance layer.

iv) Catalogue with an owner per dataset; access control on PII (also quality checks, Iceberg/Delta Lake versioning).

(b) 6 marks

The hospital's team runs Lambda: nightly Spark jobs plus a Flink stream for ICU alerts. The CTO wants Kappa. State what must be true of the stream engine, one risk of migrating, and one reason Kappa may not suit them. Show how Kappa reprocesses history.

Model answer

Must be true: the engine replays history about as fast as it handles live data, and full history is kept in a replayable log (Kafka long retention or Iceberg on S3). Risk: replaying TB+ of records may take days, and all batch jobs must be rewritten. Not suitable if: health regulators require a separate batch reconciliation, or batch SQL is preferred. Reprocessing: deploy new job version → start at offset 0 → catch up to live → swap in, retire the old job.

Question 5 Week 9, 10 marks

(a) 4 marks

A ride-hailing company in Chattogram shows a live map of 2 million driver GPS pings. The scatter is a solid blob. A filter on the 80-billion-row trip table in Hive takes minutes.

i) Name the problem with the map.

ii) Which fix shows the count of pings per area?

iii) Name the problem with the filter.

iv) Which fix gives sub-second filters, and its trade-off?

Model answer

i) Overplotting.

ii) Hexbin aggregation.

iii) Latency.

iv) A pre-aggregated data cube; data can be up to 24 h stale.

(b) 6 marks

The same company's report shows trips per district as 3-D bars with a gradient, a one-series legend, and a y-axis starting at 900. Trips rose from 1,000 to 1,050, but the bar grew from 100 px to 150 px.

i) Name two chartjunk elements.

ii) Compute the effect in the data.

iii) Compute the effect shown.

iv) Compute the lie factor and say what it means.

Model answer

i) 3-D bars, gradient fill (also the single-series legend).

ii) (1,050 − 1,000) ÷ 1,000 = **5%**.

iii) (150 − 100) ÷ 100 = **50%**.

iv) 50 ÷ 5 = **10**: the chart overstates the rise ten times; start y at 0.

OR

(a) 4 marks

A news site builds a live election dashboard. It opens on a national map, has a party dropdown, and shows a constituency's full result on click.

i) Which design rule does this follow?

ii) What is the dropdown?

iii) What is the click?

iv) Data type of constituency results on a map?

Model answer

i) Shneiderman's mantra.

ii) Zoom and filter (dynamic query).

iii) Details on demand.

iv) 2-D (map).

(b) 6 marks

Votes arrive at 2,000 events per second through Kafka. Results must refresh in under a second on the public page, and every 5 seconds on the operations screen.

i) Events in a 1-minute tumbling window?

ii) Events between two 5 s refreshes?

iii) Tool for the ops screen, and why?

iv) Delivery method for the public page?

Model answer

i) 2,000 × 60 = **120,000**.

ii) 2,000 × 5 = **10,000**.

iii) Grafana: reads the time-series DB natively and auto-refreshes every 5 s.

iv) WebSocket push to a D3.js chart.

Question 6

Week 10, 10 marks

(a)

4 marks

Dhaka's health office streams Bangla Facebook posts and pharmacy paracetamol sales into a cluster to catch dengue early.

i) Which tool should carry both event streams? (1)

ii) A dengue phrase rises from 20 to 6,000 posts an hour. What is the spike ratio? (1)

iii) A post scores a VADER compound of 0.04. Which class? (1)

iv) Why must this system not replace DGHS reports? (1)

Model answer

i) Kafka.

ii) 6,000 / 20 = **300×**.

iii) Neutral (between −0.05 and 0.05).

iv) Proxy signals reflect behaviour, not cases; Google Flu Trends over-predicted by 2×. They supplement clinical reporting.

(b)

6 marks

A hospital group forecasts weekly admissions for 400 wards with Prophet.

i) Write the Prophet model and name each term. (2)

ii) In an Eid week, g = 80, s = +6, h = +12, ε = 0. Forecast? (1)

iii) Each ward's model takes 2 s; Spark fits 20 at a time. Sequential time, parallel time? (1)

iv) The model missed a sudden COVID wave. Which term should catch it, and which hyperparameter controls that? (2)

Model answer

i) $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$: trend, seasonality, holidays, noise.

ii) 80 + 6 + 12 + 0 = **98**.

iii) 400 × 2 = **800 s**; 800 / 20 = **40 s** (20×).

iv) Trend g(t) via changepoints; `changepoint_prior_scale` (raise from 0.05 to bend faster).

OR

(a)

4 marks

DGHS wants one sepsis model from 30 hospitals and a public dashboard of cases by upazila.

i) How can it train without pooling records? (1)

ii) How can it stop small counts revealing a patient? (1)

iii) Which US law requires removing 18 identifiers? (1)

iv) A linked genomics study has 5,000 patients at 200 GB each. Storage in PB? (1)

Model answer

i) Federated learning.

ii) Differential privacy.

iii) HIPAA.

iv) 5,000 × 200 = 1,000,000 GB = **1 PB**.

(b) 6 marks

A researcher studies a misinformation cascade on X.

i) Give the steps of Louvain for finding the groups that spread it. (2)

ii) Average out-degree is 40. Upper bound on users reached in 2 hops? (1)

iii) Why is that an overestimate? (1)

iv) Kwak et al. found 22.1% reciprocity. What does that say about Twitter compared with Facebook? (2)

Model answer

i) Each node alone; move nodes to the neighbour community with the biggest modularity gain; collapse to super-nodes; repeat until no gain.

ii) 40 + 1,600 = **1,640**.

iii) Followers overlap (clustering), so people are counted twice, and not everyone reshares.

iv) Most follows are one-way, so Twitter is a directed news medium; Facebook friendship is mutual (≈100%), a social network.

For the last ten minutes

Must-know list and formula sheet

The must-know list first, then every formula from weeks 5–10. Each topic's one-line answer sits at the top of the topic; you can run through all 36 at the end.

Must-know list

YARN: RM = one per cluster (Scheduler + ApplicationsManager); NM = one per node (containers, heartbeat); AM = one per application (requests containers). Hadoop 2.0 (2013) replaced the JobTracker (MR only, ~4,000 nodes).

Container = {vCPU, RAM} slice on one node; NM kills a container that exceeds its memory limit.

YARN failures: NM -> missed heartbeat, containers marked failed, AM re-requests; AM -> RM restarts it (max attempts, default 2); RM -> HA with ZooKeeper (active + standby).

Schedulers: FIFO (starves), Capacity (guaranteed queue %, borrows idle capacity), Fair (equal shares, short jobs finish fast). DRF = fairness over CPU and RAM.

Containers per queue = min(queue RAM / container RAM, queue vCPU / container vCPU); the smaller one is the bottleneck.

Columnar: read = D x k/N; 10 TB, 3 of 100 columns -> 300 GB, 97% less, ~33x.

Parquet: row groups (~128 MB) -> column chunks -> pages; footer min/max stats -> predicate pushdown skips row groups.

Avro = row, schema evolution (reader/writer schema), Kafka ingest.

Parquet = columnar, Spark analytics.

ORC = columnar, Hive, highest compression.

Gzip is NOT splittable (1 Mapper per file).

Bzip2 splittable; LZO splittable with index; Snappy fast (splittable inside Parquet row groups).

Spark beats MR on iteration: MR = k x (read + write) on HDFS; Spark reads once and caches. 50 GB x 100 iterations: 10 TB vs 50 GB.

Spark architecture: Driver (SparkContext/SparkSession, builds DAG) -> Cluster Manager (YARN/K8s/Mesos/Standalone) -> Executors (tasks + cache).

RDD = read-only, partitioned collection created from stable storage or other RDDs; immutable, partitioned, lazy, lineage, resilient (recompute, not replicate).

Transformations (lazy): map, filter, flatMap, reduceByKey, groupByKey, join, distinct. Actions: count, collect, first, take, reduce, saveAsTextFile, foreach.

Lazy evaluation lets Spark pipeline narrow steps and stop early: filter(x > 1000).first().

Narrow (1 parent partition, same stage): map, filter, union, coalesce. Wide (shuffle, new stage): groupByKey, reduceByKey, join, repartition, distinct, sortByKey.

---

reduceByKey > groupByKey: it combines locally before the shuffle. Hash partitioner = key.hashCode() % numPartitions.

DataFrame = named columns + schema, Catalyst optimiser. HBase is NOT part of the Spark ecosystem.

Hive: HiveServer2 (JDBC/ODBC) -> Driver (parse -> semantic -> logical -> optimise -> physical) -> engine MR/Tez/Spark; Metastore (MySQL/PostgreSQL) holds schemas, partitions + HDFS paths, SerDe, stats.

Schema-on-read (Hive): schema applied at query, bad value -> NULL, fast load. Schema-on-write (RDBMS): rejected at INSERT. HiveQL: limited ACID (ORC transactional only), minutes of latency. Lecturer's plan: Hive vs SQL is an important exam topic.

Partitioning = HDFS sub-directory per value, prunes scans; low-cardinality columns only (user_id -> partition explosion). Dynamic insert: partition columns last in SELECT.	Map-side aggregation: 50 M rows, 200 countries -> 250,000x less shuffle.
Bucketing = hash(col) % N files; bucket map join (same key, same N) avoids a shuffle. Fraction read = partitions read / total.	Presto: in-memory pipeline (no intermediate disk), vectorised 1,024-row batches, cost-based optimiser, connectors.
	Point-to-point ETL = N(N-1) jobs (50 -> 2,450); bus = N connections.

Sqoop = bulk RDBMS <-> HDFS via a MapReduce job whose map tasks each read a primary-key range, incremental last-value.

Flume = Source -> Channel (memory/file) -> Sink, logs to HDFS.

Kafka = distributed, partitioned, replicated commit log. Topic -> partitions (ordered, same key -> same partition) -> offsets; consumer tracks its own offset; retention (e.g. 7 days) -> replay from offset 0.

RF = R survives R - 1 broker failures; one leader + R - 1 followers. In a consumer group, each partition goes to exactly one consumer (extra consumers idle).

Kafka speed: sequential appends (~600 vs ~100 MB/s), zero-copy sendfile (4 -> 2 copies), batching, consumer pull. 50.5 MB/s / 100 B = 505,000 msg/s.

Lambda (Marz): Batch (HDFS, accurate, hours) + Speed (stream, seconds, approximate) + Serving (merge). Weakness: two code paths (30-40% effort).

Kappa (Kreps): one stream code path over a durable log; reprocess = new job from offset 0, catch up, swap. Not for very slow TB-scale replay, batch-SQL preference, or mandated batch reconciliation.

Data lake = raw data, any format, schema-on-read; layers: ingestion, storage, processing, catalogue/governance. No governance -> data swamp.

Tufte: data-ink ratio = data ink / total ink (maximise); chartjunk (3-D, gradients, heavy grids, ducks); small multiples; sparklines; lie factor = effect shown / effect in data (1 = truthful).

Shneiderman: 'Overview first, zoom and filter, then details on demand'; 7 data types: 1-D, 2-D, 3-D, temporal, multi-dimensional, tree, network.

Chart choice: temporal -> line; categories -> bar (pie only if 4 or fewer and part-of-whole); geography -> choropleth; distribution -> histogram/box; correlation -> scatter.

Big-data viz challenges: overplotting (alpha -> hexbin -> KDE), latency (data cube / AQP, trade-off staleness), streaming (Kafka -> Spark window -> TSDB -> Grafana poll / WebSocket push).

Tools: Grafana = real-time ops; Tableau/Power BI = business BI; D3.js = custom web SVG; Superset = open-source BI on lakes; Kibana = Elasticsearch logs.

Time series: order matters, autocorrelation, trend, seasonality, non-stationarity. ARIMA(p, d, q) = AR lags, differencing, MA errors; needs stationarity and manual tuning.

Prophet: y = g(t) + s(t) + h(t) + e(t); changepoints in g (changepoint_prior_scale); Fourier seasonality; holidays; scale with groupBy + applyInPandas.

Social: Louvain maximises modularity (merge, aggregate, repeat); influence maximisation NP-hard, greedy (1 - 1/e); Kwak: 22.1% reciprocity, 4.12 hops, power law -> Twitter is a news medium. VADER: > 0.05 positive, < -0.05 negative.

Health: EHR, genomics (~200 GB/genome), wearables, DICOM (unstructured); HIPAA 18 identifiers, GDPR, PDPA; DP and federated learning. GFT over-predicted 2x: syndromic data supplements, never replaces.

▶ Short on time? 10 things you can skip

Week 5 formulas

Columnar data read

```
Read = D × k / N
```

D table size, k columns selected, N total columns

Columnar I/O reduction

```
Reduction = 1 - k / N; speedup ≈ N / k
```

10 TB, 3 of 100: 97%, 33×

Pushdown saving

```
Speedup = D / R
```

R = bytes actually read; 10 TB / 50 GB = 200×

Mappers per file

```
Splits = ⌈F / B⌉, or 1 if gzip
```

B = block or row group, about 128 MB

Containers per queue

```
min(nodes·f·RAM / c_ram, nodes·f·CPU / c_cpu)
```

smaller term is the bottleneck

Nodes for a job

```
⌈T / min(RAM / c_ram, CPU / c_cpu)⌉
```

T containers needed at once

Extra work from utilisation

```
U_new / U_old - 1
```

78/45 – 1 = 73%

Week 6 formulas

MapReduce iterative disk I/O

```
MR_IO = k × (D_read + D_write)
```

k iterations; slide: 100 × (50 + 50) GB = 10 TB

Spark HDFS I/O

```
Spark_IO = D (read once)
```

later iterations read RAM, not HDFS

Transfer time

```
t = size / speed
```

50,000 MB / 500 MB/s = 100 s

Iterative wall-clock time

```
T_MR = k × (D/R_hdfs + D/W_hdfs);
T_Spark = D/R_hdfs + (k - 1) × D/R_ram
```

k – 1 RAM passes, not k

Speedup

```
Speedup = T_old / T_new
```

A1: 20,000 / 1,090 ≈ 18.3×; RDD paper 127 / 0.96 ≈ 132×

Hash partitioner

```
partition = key.hashCode() % numPartitions
```

17 % 4 = 1

Shuffle records

```
groupByKey = all pairs; reduceByKey ≤ partitions × distinct keys per partition
```

4,000 vs 200 records

Efficiency per node

```
(T_old × N_old) / (T_new × N_new)
```

GraySort (72 × 2,100) / (23 × 206) ≈ 32×

Week 7 formulas

Map-side aggregation

```
reduction = N_rows / G (with M mappers: N_rows / (M × G))
```

50M rows, 200 countries: 250,000×

Partition pruning

```
fraction read = P_read / P_total
```

P_total = product of values per partition column

Bucket number

```
bucket = hash(column) % N
```

integer key: hash = value; 98 % 4 = 2

Bucket file size

```
size = S / (P × N); files = P × N
```

aim near the 128 MB HDFS block

ALS model size

```
(users + items) × k × bytes
```

(75M + 30M) × 50 × 4 B = 21 GB

Speed-up

```
T_old / T_new; saved = (T_old - T_new) / T_old
```

8 h to 2 h: 4×, 75% saved

Pairs in a playlist

```
L(L - 1) / 2
```

20 songs: 190 pairs

Week 8 formulas

Point-to-point ETL jobs

```
N × (N - 1); bus = N
```

50 systems: 2,450 jobs vs 50 connections

Partition copies

```
P × R
```

bkash-txns: 3 × 2 = 6

Broker failures survived

```
R - 1
```

default R = 3 survives 2

Active consumers in a group

```
min(P, C); idle = C - P (only if C > P)
```

one consumer per partition per group

Messages per second

```
MB/s × 1,000,000 ÷ message bytes
```

50.5 MB/s, 100 B = 505,000 msgs/s

Speedup

```
Kafka ÷ other
```

50.5 / 4.3 ≈ 11.7×; 50.5 / 14.7 ≈ 3.4×

Sequential vs random disk

```
seq ÷ random
```

600 / 100 = 6× (paper figures)

Zero-copy

```
4 copies → 2 copies
```

about 2× throughput, CPU near zero

Events per second

```
events per day ÷ 86,400
```

Netflix 100 B/day ≈ 1.16 M/s

Week 9 formulas

Data-ink ratio

```
ink encoding data ÷ total ink
```

Maximise it; 1 is the ceiling

Lie factor

```
effect shown in graphic ÷ effect in data
```

1 truthful; above 1 overstates; below 1 understates

Hexbin reduction

```
points ÷ hex cells
```

10 M ÷ 10,000 = 1,000×

Alpha stacking

$$opacity = 1 - (1 - \alpha)^n$$

α = 0.02: n = 100 → 0.867, saturates

Cube rows

$$\leq d_1 \times d_2 \times \ldots \times d_k$$; reduction = fact rows ÷ cube rows

100 B ÷ 5 M = 20,000×

Group-by combinations

$$2^k$$ for k dimensions

k = 4 → 16

Events per window

```
rate × window seconds
```

10,000/s × 60 = 600,000

Updates per day

```
86,400 ÷ refresh seconds
```

Every 5 min → 288

Week 10 formulas

Prophet model

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

trend + seasonality + holidays + noise

Prophet trend

$$g(t) = (k + a(t)^T\delta)\ t + (m + a(t)^T\gamma)$$

δ slope changes at changepoints, sparse prior

Fourier seasonality

$$s(t) = \Sigma\ [a_n \cos(2\pi nt/P) + b_n \sin(2\pi nt/P)],\ n = 1..N$$

P = 365.25 yearly, 7 weekly

ARIMA(p, d, q)

```
AR past p values; I difference d times; MA past q errors
```

needs stationary input

Speedup

```
T_sequential / T_parallel
```

480 min / 12 min = 40× (5,000 SKUs)

Reach in h hops

```
d + d² + … + d^h
```

upper bound; d = 45, h = 3 gives 93,195

Greedy influence bound

```
spread ≥ (1 - 1/e) × optimal ≈ 0.632 × optimal
```

Kempe et al. 2003

Modularity (words)

```
fraction of edges inside communities - fraction expected by chance
```

Louvain maximises it

Power law

```
P(k) ∝ k^(-α), α ≈ 2.276
```

Twitter follower counts (Kwak 2010)

Spike ratio

```
current rate / baseline rate
```

10 to 10,000 per hour = 1,000×

VADER classes

```
> 0.05 positive; < -0.05 negative; else neutral
```

compound in [−1, +1]

STL anomaly rule

```
|residual| > k × IQR, IQR = Q3 - Q1
```

IQR of residuals, not raw data

Genomics storage

```
200 GB × patients (× replication)
```

10,000 patients = 2 PB

Every topic in one line